Everything but the Software:

Feature-Driven Decay in an Agentic Coding Loop. An Exploratory Case Study and an Experiment Protocol

Juan Manuel Servera Bondroit*

Senior Partner Solution Architect, Microsoft Schweiz, Zurich

Draft of 29 October 2026

Abstract

Each feature changes the starting point for the next. In agentic software development, that starting point includes code, tests and the prompts, skills and notes directing subsequent work. This paper examines how faults and corrective instructions can accumulate across this process. Industrial maintenance accounts, recent agent benchmarks and public project records motivate a distinction between errors within a conversation and errors inherited through a repository. A paired-history benchmark supplies evidence of the latter, while its clean-history results qualify an explanation based on structural complexity alone. A six-week architectural demonstrator provides a motivating testbed: 183 pull requests were merged in ten days, while the replacement remained unusable for the participant. Five candidate mechanisms organise its delivery record: specification inflation, proxy capture, learning evaporation, process-gate inflation and feedback starvation. The case centres on a migration with missing deployment dependencies; it exposes a break in usable behaviour rather than measuring a decay rate. A prospective protocol specifies an exploratory, registration-pending twelve-trajectory pilot comparing feature-only delivery, scheduled consolidation and read-only review. Its principal outcomes are final behavioural coverage, retained-contract loss, an unforeseen change request and resource cost. The consolidation arm includes additional resources, tests and a different handover note. The pilot has not started; no experimental results are reported. The pilot asks whether consolidation improves the system inherited by later feature work.

Keywords: agentic software engineering; coding agents; spec-driven development; software evolution; technical debt; refactoring; feedback loops; human–AI interaction; exploratory case study; experiment protocol.

1 Introduction

On 1 August 2012, Knight Capital's automated router sent more than four million orders while attempting to fill 212 customer orders. The firm lost more than USD 460 million. The SEC traced the incident to a defective function left in the router after a 2005 change, an incorrect deployment for a new trading programme, and inadequate controls. The new deployment activated old code that was no longer meant to run [82]. The failure crossed a feature boundary and a seven-year interval: the relevant starting point was the accumulated system, not just the latest patch.

Less spectacular versions of the same maintenance problem are familiar. Cunningham's 1992 account of WyCash described a financial product continually adapted to market demands. Its

*The views expressed are the author's own and do not necessarily reflect those of Microsoft. The case organisation, Zava North Homes, is fictional.

mature code was repeatedly revised and rewritten; the technical-debt metaphor explained why taking an initial shortcut required later consolidation [27]. Some debt is visible as duplication or awkward dependencies. Some remains hidden in assumptions about inputs, deployment and earlier behaviour until another change depends on it. Counting completed changes reveals neither.

Agentic development makes this distinction newly pressing. PhotoCraft, an early-alpha reimplementation of Photoshop, reports 631 of 631 menu items live, but explicitly defines that count as dispatch rather than behaviour. Its own dated assessment estimates about 76% feature breadth and 45% readiness for real work [78]. A command can be reachable without the surrounding workflow being dependable. At the repository level, SWE-Maintain finds that agents solve fewer later tasks when starting from their own earlier changes than from the developers' changes, even though each task starts in a fresh session [74]. The next session can forget the conversation and still inherit its mistakes.

The engineering response has a long history: isolation and explicit contracts limit dependencies; tests and continuous integration expose regressions; small releases shorten feedback; refactoring preserves room for later change [8, 9, 33, 75]. Recent discussions by Beck, Martin and Fowler revisit these concerns for agent-assisted work [34, 60]. Beck's distinction between producing code and learning through working software sharpens the research question [15, 72]: what does a feature leave behind for the next one?

The question arose during an Azure Master Architect (AMA) capstone, used here as an exploratory testbed rather than the scope of the contribution. After intermittent use of an MVP, its sole developer started a coordinator on 15 October 2026 to deliver more than 180 backlog issues. Ten days and 183 merged pull requests later, the developer could not sign in to the replacement. The public repository, pull requests, append-only log and versioned coordinator instructions provide a trace of that transition [83]. The case motivates hypotheses, not a general estimate of agent-caused deterioration. The author's earlier Aithena experience and the PhotoCraft inspection selected after the AMA analysis have separate provenance in the ancillary anc/supplementary-context.md.

The working hypothesis is that feature-driven delivery can carry unresolved errors and avoidable change burden forward unless it includes cumulative verification and consolidation. Two kinds of state matter: the software being changed, and the instructions used to change it. Adding a corrective rule after every failure can leave prompts, skills and handover notes with their own maintenance burden. A fresh agent inherits both kinds of state. Section 2 distinguishes these pathways; the case then examines how they interact with completion proxies, review capacity and product-use feedback.

Here *feature-driven decay* describes loss of retained behaviour or increasing change burden across feature steps. Whether feature-driven decay occurs remains a question for the prospective study.

Three questions guide the investigation:

- **RQ1.** Through which retained state can errors and change burden carry from one feature to the next, and what does existing evidence show about those pathways?
- **RQ2.** How does an exploratory delivery record illuminate that problem, and which candidate mechanisms and rival explanations does the gap between completion signals and usable replacement behaviour suggest?
- **RQ3.** Does scheduled agent-led consolidation after each feature improve retained behaviour and future changeability, and at what cost?

The report makes three linked contributions:

1. A focused synthesis distinguishing conversational self-conditioning, repository-carried faults and instruction maintenance, with a replot of published paired-history data (Section 2).

2. A documented six-week case, including a ten-day autonomous delivery loop, and an integrated account of five mechanisms. Descriptive measures, within-case contrasts and an evidence audit connect the observed interruption of usability to competing explanations (Sections 3–5).
3. A consolidation harness and a twelve-trajectory pilot protocol to estimate retained behaviour, future changeability and resource cost. It follows the registered-report example of Lulla et al. [58] and HVE Core's Minimum Viable Experiment method (Sections 6 and 7).

The case concerns a migration and a configuration designed by the author, including delegated delivery and merge decisions. The case suggests hypotheses whose causal relationships remain to be tested. The planned pilot estimates a multi-component, extra-resource package comparison; it does not isolate refactoring or validate the five case mechanisms individually. It is not running, and registration and launch checks remain pending. The case's validity and the pilot's identification limits are consolidated in Sections 5.7 and 7.6.

2 Background

2.1 Software that only grows

That software decays when it only accumulates features is one of the oldest observations in software engineering. Lehman's laws of software evolution hold that a program in real-world use must keep changing, and that its complexity grows as it evolves unless work is done to maintain or reduce it [54]; a later formulation adds that its quality will appear to decline unless it is rigorously maintained and adapted [55]. Parnas called the result software aging, and blamed in part the changes made by people who do not understand the original design [75]. Eick and colleagues found mixed but, on the whole, persuasive evidence of code decay in fifteen years of change data from a large telephone-switching system [32]. Cunningham's technical-debt metaphor framed the cost of not consolidating as interest paid on every later change [27], and Fowler's design stamina hypothesis holds that without attention to design, adding features eventually becomes slower than it would have been with it [33]. Studies have since put numbers on the interest: across 39 proprietary code bases, low-quality code had fifteen times more defects and took 124% longer to develop than high-quality code [89], and developers in a longitudinal study lost about 23% of their working time to technical debt [17].

The evidence is not universal. Increasing complexity and declining quality are among the less strongly supported of Lehman's laws, and some systems have grown without the predicted decay [42]. Several findings above are observational, and their estimates belong to the studied settings, not every code base.

The engineering intuition often called *software entropy* is that keeping a changing system coherent requires counter-work: removing obsolete paths, restoring boundaries and revisiting assumptions. Software does not deteriorate simply because time passes, and thermodynamic entropy supplies no measure of its quality. Here the measurable quantities are losses of required behaviour, excess structural burden and the effort needed for a later change.

2.2 What carries from one feature to the next?

Feature accumulation is the addition of capabilities and interactions; churn is the editing activity that implements, replaces or removes them. Neither is itself deterioration. A *regression* loses previously working behaviour that is still required. A never-implemented feature is instead an acquisition failure. Persistence means that the same fault remains unresolved; propagation means that a later, distinct fault or failure can be traced to reuse of the earlier one. *Compounding* is the stronger claim that the earlier error increases subsequent error risk or change burden.

These distinctions matter because an agent inherits more than a code diff. The repository supplies implementations, tests, interfaces and deployment assumptions. Instructions, skills and notes supply an account of how to work on them. A mistaken rule can become code, then a test expectation or an instruction, and finally the precedent used for another feature. The stored record need not be correct to become authoritative in the next session. Conversely, an inherited implementation can be sound even when it differs from a human reference. The question is what subsequent work can achieve from that state.

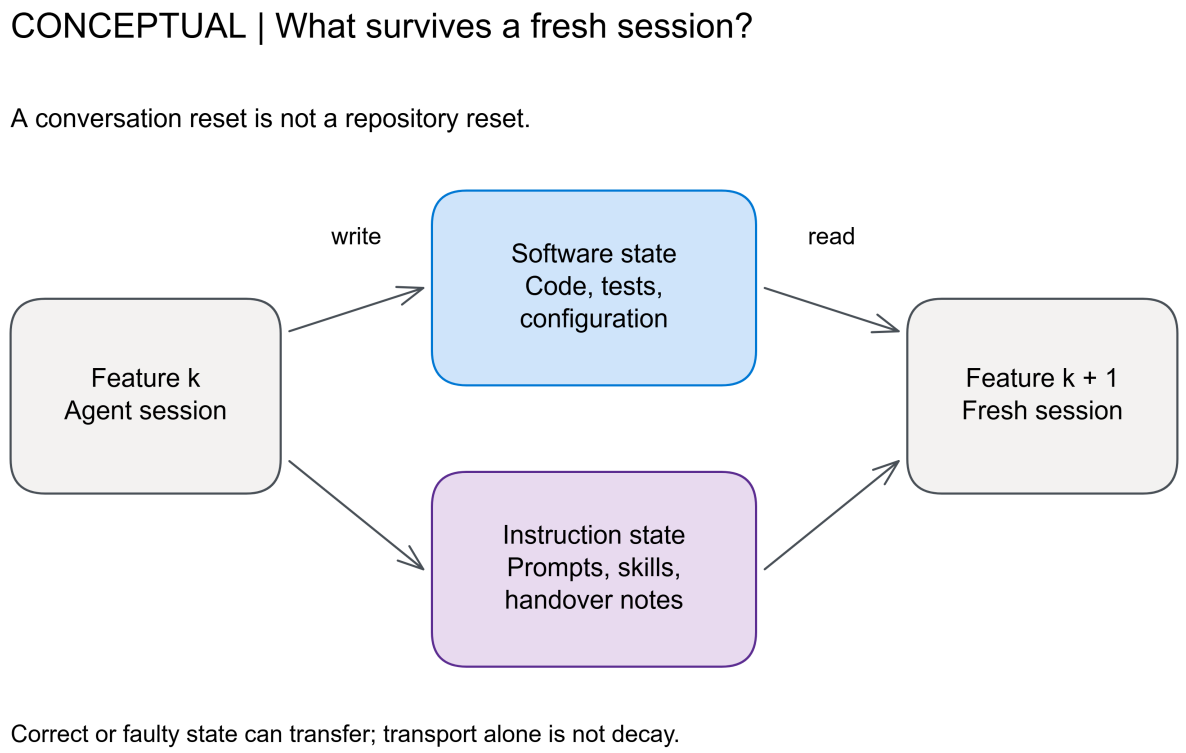


Figure 1: Conceptual state transfer between feature sessions. A fresh conversation does not reset inherited software or instruction state. Solid arrows denote possible read/write paths, not measured error propagation. Both channels can carry correct decisions as well as faults; their presence alone is not evidence of decay.

Figure 1 separates the two persistent channels from conversation history. It is an explanatory model, not an observed trajectory or a claim that every tool retains the same material.

2.3 Compounding within and between features

Errors carried in conversation. An earlier answer changes the context from which a model produces the next one. Experiments on this pathway distinguish dependence on an erroneous history from the difficulty of a long task.

Zhang et al.'s earlier *How Language Model Hallucinations Can Snowball* shows why consistency is not enough. On three question-answering datasets, ChatGPT and GPT-4 produce false justifications after wrong answers, yet often reject the individual false claims when questioned in a separate session. Their explanation is over-commitment to the earlier answer rather than only a lack of knowledge. The experiments concern question answering in two 2023 models [95, Secs. 3.1–3.4].

Sinha et al. test the distinction directly in a controlled execution task. Models receive both the lookup dictionary and the plan, then maintain a running sum. The authors vary the fraction of incorrect answers in artificial prior histories and measure the accuracy of subsequent state

updates, not merely whether the final total remains correct. More erroneous history reduces later accuracy beyond the effect of context length. This is counterfactual evidence of *self-conditioning*: earlier mistakes can increase the chance of new mistakes. Scaling non-thinking models does not remove the effect, but Qwen3 models with thinking enabled avoid it in this setup. The result identifies a dependence on model configuration as well as erroneous history [86, Sec. 3.2, Figs. 5–6].

Laban et al. supply complementary conversational evidence. Across six generation tasks, including Python and SQL, they report a 39% average relative performance drop when instructions arrive through underspecified multi-turn conversation rather than all at once. A concatenated-instruction control largely preserves performance, weakening information loss from rephrasing as the explanation. Their analyses associate premature answers with lower success and describe reliance on earlier incorrect attempts and increasingly bloated solutions. These are simulated conversations and diagnostic analyses, not a controlled estimate of the cost of a particular inherited coding defect; the 39% is not a percentage-point loss or a repository defect rate [52, Secs. 3.3 and 6.1, App. F.1–F.2].

Errors carried in the repository. Pan et al.'s *SWE-Maintain* preprint tests the pathway most directly relevant to successive feature work. It reconstructs 100 three-event maintenance episodes from 50 Python repositories and runs each later task from two starting points: the developers' preceding patches and the agent's own preceding changes. Task, agent and tests are held fixed, and every event uses a fresh session: only the repository carries the earlier work forward. Across 1,996 paired later events, agent history lowers new-task resolution by 5.6 percentage points (95% interval $[-7.2, -4.1]$). Requiring earlier tests to remain passing widens the gap to 14.0 points. The average new-task gap grows from 4.0 points at the second event to 7.2 at the third, consistent with an accumulating downstream cost, though three-event episodes cannot establish a general long-horizon decay law [74, Secs. 2.5 and 4.1, Table 2].

Their `python_unidiff` example makes the proposed propagation concrete: an agent first splits an input string into lines, losing line terminators; on the next task it reuses that defective logic in the constructor rather than repairing it. The same later task succeeds from developer history. Their `parse_crontab` example exposes a less visible dependency: an agent's local Python 3 fix passes the current and next tests, but later code relies on module-wide compatibility aliases that it never supplied. The authors report eight of ten later tests failing with undefined names under agent history [74, Sec. 5.1, Figs. 4–5]. Among the 619 later events whose inherited code passes earlier tests and whose next task is not already solved, the event-resolution difference is +2.7 points (95% interval $[-0.5, +6.1]$), with no detected disadvantage. This is a selected clean-history subset, not a randomised repair intervention, and it does not prove equivalent maintainability under all changes. It points more directly to inherited behavioural failures than to structural complexity as the main observed channel [74, Sec. 4.2, Table 3; Sec. 5.1, Fig. 5].

Figure 2 shows the published event-resolution contrasts, including the clean-history cohorts. Event resolution requires the new task and all earlier non-superseded tests to pass. Its larger gap therefore includes retained failures as well as difficulty solving the new task. The current-task-only difference of −5.6 points is important for the cross-feature hypothesis precisely because it does not count earlier failing tests against success on the new task.

The conversation and repository studies therefore locate two different transmission pathways. Resetting context addresses the first, but not faulty code retained for the next feature. SlopCodeBench and SWE-CI extend the evaluation horizon to iterative structural erosion and behavioural regressions [24, 71]. A randomised maintainability study found no significant downstream difference between AI-assisted and unassisted source code [20]. The emerging question is which inherited failures survive, how they affect later work, and which interventions prevent or repair them.

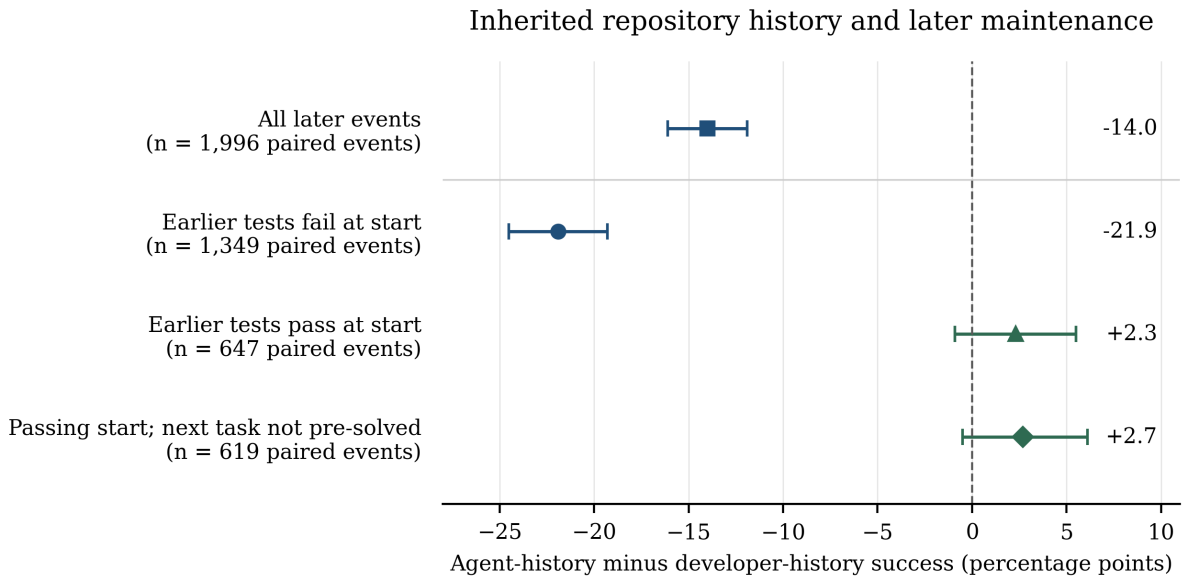


Figure 2: Later maintenance from two repository histories. Points and 95% repository-clustered bootstrap intervals are transcribed from Pan et al., Table 3 [74]. The outcome is event resolution: current and all earlier non-superseded hidden tests pass. Negative differences favour developer history. The 1,996 paired events come from 100 three-event episodes in 50 Python repositories, evaluated with five systems and two models. Subgroups overlap and are selected by the agent-history starting state; the last excludes pre-solved and unevaluable tasks. They are not randomised repair conditions.

Output and review capacity. Understanding changes is a major challenge of code review; habituation, fragmented context and uneven retrieval from long contexts may limit scrutiny as output grows [4, 22, 43, 57, 94]. Faster feature delivery also shortens the time available to discover that an earlier assumption has become the foundation of another change. Further empirical and practitioner context is retained in the ancillary supplement.

2.4 Corrective instructions as maintenance debt

A failed agent action often invites a new instruction: run this check, never edit that file, remember this exception. GitHub's account of more than 2,500 agent-definition files recommends starting small, testing and adding detail when an agent makes mistakes [69]. The resulting guidance can preserve useful lessons, but it also becomes another maintained system. Chatlatanagulchai et al. identify *context debt* as a maintenance problem for this material [23, Sec. 5.1]. Here *instruction debt* denotes the avoidable burden of obsolete, duplicated, conflicting or poorly routed directives in prompts, skills and handover notes. This extends the maintenance question from the product to the machinery that changes it.

Repository histories show that this material evolves. Mohsenimofidi et al. found no post-creation change in 77 of 155 instruction files. In the ten most frequently changed files, however, they classified 169 commits: 78 added instructions, 59 modified them and 23 removed them; change categories could overlap [65, Sec. 4.3, Table 2]. Chatlatanagulchai et al.'s manual analysis of 100 post-adoption commits produced 78 addition, 32 modification and 19 deletion labels; addition labels occurred across 66 distinct commits. Figure 3 replots the label counts [23, Secs. 4.2.2–4.2.3]. These histories contain accumulation and pruning, rather than a uniform march towards longer files. The authors associate their day-scale edit intervals with ongoing project maintenance rather than immediate prompt debugging.

Gloaguen et al. test the effect of supplying such context. Across four agent–model pairings on 300 SWE-bench Lite tasks and 138 CTXbench tasks, LLM-generated context files increased average inference cost by 20% and 23%, respectively, without a statistically significant improvement

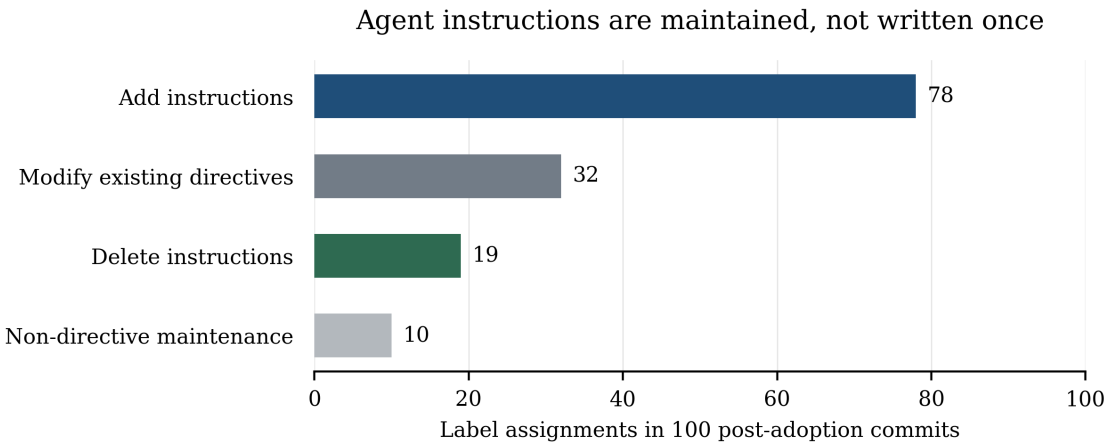


Figure 3: Instruction-file maintenance in 100 randomly sampled post-adoption commits, from Chatlatanagulchai et al., Sections 4.2.2–4.2.3 [23]. Commits can receive several labels; line-level and section-level add/delete labels are combined. The ten non-directive maintenance labels are the remainder of 139 total assignments. The chart counts edit types, not net token growth or effects on agent performance.

in task resolution. Agents followed the instructions, performed more testing and exploration, and spent more reasoning tokens. Yet file length did not explain success, and generated context helped when other documentation was removed [38, Secs. 4.2–4.4, App. B]. Instruction count and loaded tokens are therefore different exposures from the obligations and behaviours those instructions induce.

Anthropic's context-engineering account describes the practical tension: provide enough information to guide action, but avoid brittle prompt logic and an expanding catalogue of exceptions [79]. Maintaining these instructions means preserving necessary constraints while replacing accumulated exceptions with coherent rules, executable checks and task-specific retrieval. To test the corrective-rule spiral directly would require versioned instructions, actual session inputs and outcomes across successive revisions. The testbed record supplies document growth and rule changes; the consolidation protocol tests a bounded form of handover maintenance by replacing, rather than appending to, its design note.

SkillOpt supplies counterevidence to treating instruction change as inherently harmful. Its first-party account describes bounded edits to skill files, held-out validation and retention of successful versions without changing the target model's weights [64]. Procedural knowledge needs evaluation and maintenance; its length alone tells us little about its quality.

Jiang et al.'s preprint compares workflow memory with skills distilled from the same execution histories. Its paired trajectory analysis associates skills chiefly with more reliable procedures, while also identifying misapplication and insufficient runtime verification [47, Secs. 3.3, 5.1–5.3]. This qualifies an instruction-debt account: retained guidance can stabilise execution rather than merely burden context. Their retrieval study also separates skill access from task success; in their comparison, lower exact-match access was observed alongside sustained downstream success [47, Sec. 5.5]. Because retrieval diagnostics are run separately from task execution, this comparison does not isolate the effect of reduced skill access. Neither result measures the effects of accumulating repository instructions across feature changes.

2.5 Refactoring, review and evaluation boundaries

Agentic refactoring, review and human correction are already studied. RefactorBench evaluates 100 handcrafted multi-file refactoring tasks across nine Python repositories, not scheduled consolidation between feature additions [37]. Kumar et al.'s ASE 2025 study observed developers collaborating with agents on existing issues: incremental, active collaboration was associated

with greater success, not randomly assigned to establish an optimal intervention rate [51]. SWE-Sharp-Bench supplies a reproducible C# issue-resolution benchmark and curation pipeline [61]; reproducible repository-scale evaluation itself is not a contribution claimed here.

Two recent preprints sharpen the evaluation problem. CRBugBench evaluates review agents on 1,000 open-source and 800 proprietary tasks from one company, reporting weaker detection on complex, non-local and proprietary-context changes [93]. It is a 9 October 2026 preprint, not an independently replicated result establishing that review cannot work. *Building to the Test* reports near-perfect behavioural scores for demos while the requested reusable library was unused or absent. Its unchanged, source-hidden oracle did not check that delivery contract [59]. This is distinct from altering tests, as examined in ImpossibleBench [96]: test integrity and adequate outcome coverage are separate safeguards.

These studies narrow the proposed novelty to the case trace and the specific prospective package comparison, including an unforeseen change request. They also caution against assuming that another reviewer, a stricter check or a high hidden-test score necessarily certifies the intended deliverable. Preprints, observational studies, practitioner accounts and first-party industrial reports have different evidential roles; none substitutes for the planned controlled comparison. AgentRx's first-party release describes evidence-backed diagnosis of a trajectory's first critical failure [62]. It motivates traceable failure localisation. An independent diagnosis of the AMA mechanisms remains outstanding.

2.6 Optimisation with behavioural feedback

Pavlenko et al.'s *Towards an AI Software Factory for Data Systems* offers a complementary case [76]. Hotspan packages optimisation targets from telemetry, analysis and reviews; Darwin explores candidate changes through code–test–debug–reflect cycles. The setting is existing, human-architected systems with stable semantics and measurable objectives such as latency. Reported optimisation gains are evidence against treating high agent throughput as inherently unproductive, not a controlled estimate of end-to-end productivity, general maintainability or future changeability. Their application-work productivity calculation excludes infrastructure and onboarding time, and output enters engineering review. The work establishes a precedent for bounded agent-led correction; it does not isolate consolidation between feature additions or product-goal selection.

2.7 The practitioner conversation

Waseem et al. describe rapid generation alongside architectural inconsistency, security concerns and maintenance overhead [90]. That trade-off is not claimed as new here; the contribution is the case trace and prospective package comparison.

Beck supplies the conceptual bridge: augmented coding retains care for code, complexity and tests; alternating features with consolidation aims to restore options consumed by additions [12, 13, 15]. Learning through use, understanding and abstraction are distinct from producing more artefacts [14]. These arguments motivate testing bounded reflective work, not an established remedy or a permanent human–agent boundary. Extended practitioner accounts remain in the ancillary supplement.

Spec-driven development adds structure, but not an automatic outcome oracle. Spec Kit proposes specification, planning, tasks and implementation; Böckeler describes both ignored and over-followed instructions; and a study of practice reports reliability benefits alongside planning and review costs [1, 19, 28].

2.8 Engineering principles behind consolidation

The engineering lineage is more specific than a general call for better code. Beck links small structural changes to test feedback [8, 11]; Gamma, Helm, Johnson and Vlissides describe reusable

designs with explicit applicability, consequences and trade-offs [36]. Martin emphasises executable technical feedback [60], while Fowler's design stamina hypothesis connects attention to design with the cost of later features [33]. These ideas motivate the consolidation checklist: locate responsibilities, expose dependencies, preserve contracts and leave a concise account of design choices. They are design rationales, not empirical evidence that the proposed agent intervention is effective. Applying more named patterns is not itself an outcome; retained behaviour and the cost of subsequent change are the proposed tests.

Naur distinguishes a programmer's theory of the system from its documentary record; Brooks locates essential difficulty in the conceptual construct, not its representation [21, 68]. Royce and Gall motivate early experience of working behaviour [35, 80]. XP's short releases and tests [8, 9], Crystal Clear's incremental team practice [25], tracer bullets [45] and Agile's preference for working software [2] make that feedback concrete. Small reversible structural changes supply a consolidation practice [11].

Automation can shift effort from producing to monitoring and intervention, with substantial metacognitive demands [5, 85, 88]. Associations between confidence in AI and reduced scrutiny do not establish causation [53]. Goodhart and Strathern motivate checking completion proxies against their intended outcomes [40, 87].

2.9 HyperVelocity Engineering (HVE Core)

HVE Core is an open-source framework from Microsoft that supplies workflow structure for GitHub Copilot; the author's coordinator provided autonomous worker dispatch. Its Research, Plan, Implement and Review (RPI) workflow leaves dated records under `.copilot-tracking`, with explicit human stops and review. The case used those file-based handovers and its Experiment Designer's Minimum Viable Experiment method [46]; the coordinator supplied the separate autonomous delivery loop. Framework product details are secondary context, retained in the ancillary supplement. The author reports using HVE Core plugin snapshot `plugins-v3.3.106` throughout the case. Its snapshot commit is `2df5013e29ebfd2ce61bcd0cf021d8c8186482eb`, generated from source commit `e9af018fe5ffbfbc72321a8bca21fb987359af08`. Historical use is an author account; the snapshot identifiers do not establish an installation date.

3 Case and method

3.1 Setting

The motivating testbed is a capstone for Microsoft's *Azure Master Architect* (AMA) programme. The programme's name is public in the Microsoft Employee Learning badge listing.[1] That public listing does not authorise publication of internal assessment material or this paper. Case Study 48 supplies a fictional not-for-profit housing association, Zava North Homes, with about 60,000 homes. Its retrofit demonstrator combines assessment, grants, procurement, resident consent and quality assurance. The exercise required research, requirements, architecture, a delivery plan, a working demonstrator and an executive presentation. Domain obligations provide realistic dependencies; no actual housing service or resident data is evaluated here. The top rubric descriptor for implementation completeness is "all required capabilities working end to end".

The author worked alone, as architect and product owner, from 11 September to 24 October 2026. Agents did almost all of the research synthesis, writing and coding. The repository also contains an automated scorer: a GitHub agentic workflow that asks a language model to score each pull request against the rubric and records the result [83].

[1] https://www.credly.com/org/microsoft-employee-learning/badge/azure-master-architect

3.2 Workflow and tools

Interactive development used GitHub Copilot in VS Code, mainly Opus 5.5 during the first two weeks and then GPT-6.1 Sol through the case cut-off. These are recalled interface labels, not verified provider build identifiers or an exact switch date.

- **Research.** Sixteen official legislation, regulator, policy, standards and funding documents were downloaded, converted to Markdown and organised, with agent-written sector and company research, into a knowledge base: an index, immutable sources, maintained syntheses, an owner-curated memory file and an append-only log of every change.
- **Challenge.** Eight agents role-playing the case organisation's stakeholders (asset management, programme delivery, finance, procurement, data, building safety, neighbourhood housing and resident engagement) and HVE Core's Design Thinking coach challenged each idea. Their first verdict was that none of the five end-to-end agents requested in the brief should be built as such.
- **Requirements and decisions.** HVE Core's BRD and PRD builders, a BRD quality reviewer and the ADR assistant produced and revised the requirements and six architecture decision records.
- **Prototypes and MVP.** Several prototypes were built, reviewed by the role agents and kept as evidence. An MVP followed and was deployed to Azure.
- **Backlog.** HVE Core's backlog manager turned the requirements and decisions into GitHub issues with epics, features, acceptance criteria and dependencies, labelled by how much of each item already existed in the MVP.
- **Coordinator.** On 15 October the author added a custom coordinator to deliver the backlog (Section 3.3).

3.3 The coordinator

The coordinator ran through Copilot CLI on a VM. It selected waves of issues whose dependencies were merged and launched parallel worker CLI sessions through `tmux`, each in an isolated git worktree with HVE Core's RPI agent and one issue as its brief. The author reports using Sonnet 5.5 for the workers, including before the explicit pin. At the pinned repository commit the worker selector is `claude-sonnet-5.5`; PR #420 introduced that requirement on 21 October. Earlier use is recollection; the committed selector independently records the later configuration. The coordinator used Auto selection with the Intelligence setting. The author observed it tending to select GPT-6.1 Sol, but this was not a guaranteed model pin or proof of every session's actual model or provider build. Within its first day the author added four rules, each in response to a reported problem. First, it could merge its own pull requests through a gate it verified itself, because repository auto-merge was unavailable. Second, the full research, plan, implement and review loop became mandatory for every issue, and the presence of the loop's `.copilot-tracking` artefacts in the pull request became a merge condition, because the first waves of sessions had implemented directly without them. Third, it updated the project board and ticked only the acceptance criteria that reviewed evidence proved. Fourth, it monitored sessions on a schedule instead of blocking. Later rules required a complete local validation gate before every push, including documentation-only changes, covering five source projects, infrastructure templates, a disposable SQL Server, browser accessibility checks, migrations, four container images, vulnerability scans and a software bill of materials, to be rerun after every correction or rebase. An issue stopped only on a *human blocker*: an owner decision, a named approval, a missing credential, or a review finding still open after two correction cycles. On the same day an accepted ADR made migrating the working MVP into a new source layout the first priority and barred new feature code from the MVP.

3.4 Data and analysis

The study uses an exploratory single-case design, with the author as participant-observer. The case is the delivery of this repository from 11 September to 24 October 2026. The unit of analysis is the autonomous loop of 15–24 October. The loop is compared with the preceding MVP period of 8–14 October within the same case, acknowledging the uncontrolled differences between them. The author used the system intermittently during that period to refine ideas and the user experience. The case was selected because it is revelatory: a public, contemporaneous trace of an autonomous agent loop.

The analysis uses the public repository at commit `29c6b51` (24 October 2026, 16:19 UTC) [83] with issue and pull-request metadata retrieved from the GitHub API on the same day. The main sources are the append-only knowledge-base log (368 dated entries and about 57,000 words from 11 September to 22 October; entries before 25 September were reconstructed from history), the knowledge-base index, the BRD (version 0.19), the PRD (version 0.18), the coordinator's definition, issue bodies and the rubric scorer's latest result. Methods comprise descriptive counts; a hedging lexicon applied to each log entry (Appendix C); reference tracing of recorded learning identifiers through the log by date; and AI-assisted interpretation of the author's observations. These informed the candidate mechanisms, possible amplifiers and consolidation hypothesis. GitHub Copilot in VS Code organised those explanations, named the five patterns and drafted the account, using Opus 5.5 and then GPT Sol 6 and 6.1 as reported interface labels. Microsoft 365 Copilot helped summarise material and find relevant reference documents; it did not perform the log analysis or develop the mechanisms. This was not a systematic entry-by-entry qualitative coding study. The author reviewed important specification sections selectively, especially the addendum, rather than auditing the full BRD and PRD. Their identifiers and cross-references made the overall design difficult to follow. The author reports checking the repository figures and Azure expenditure. Independent author checks of numerical findings from cited studies and of the log-growth figure remain pending. The central discrepancy needs a compact, release-linked evidence package: the dated score and its assessed capability scope, the deployment revision and configuration, and dated evidence of the failed sign-in journey. The retained account does not yet establish that the scorer assessed the same release and capability claim as the usability observation. Until those links are checked, their coexistence is reported in the case chronology, not a verified same-release contradiction. The report is informed by established guidance for case study research in software engineering [81] and the ACM SIGSOFT Empirical Standards [84]. As the sole developer, the author is a participant-observer; Section 5.7 discusses what that implies.

The contextual observations in the ancillary supplement were added on 25 October after the case analysis. They use purposively selected public repositories, pinned source files, GitHub tracker counts, and ten newest plus five oldest open `bug`-labelled PhotoCraft reports. They are not a systematically sampled multi-case study and are not pooled with AMA's measurements. The ancillary repository-observation record states the retrieval boundary, locators and exclusions. No external application, original Photoshop installation or reference-parity suite was executed.

The literature-led framing was developed on 27 October. Source selection followed three questions: how earlier defects affect later changes, how agent instructions evolve, and how completion measures relate to usable behaviour. Sources comprise the SEC's Knight Capital account, the original WyCash report, the dated public repository records and the cited empirical studies. Knight Capital provides a documented inherited-fault example; PhotoCraft provides a project-authored distinction between command coverage and readiness. Their outcomes are not pooled with AMA. The two new figures replot published values rather than digitising curves or running the original systems. Versioned claims, locators and transcription details are retained in the source package README and anc/published-evidence.json. The review offers a focused narrative synthesis, with no claim to systematic coverage. The two sources added on 29 October were selected by the owner for relevance to judgment and skill reuse, not through an updated systematic search.

3.5 Constructs and evidential boundaries

Here an *agent* is a configured model–tool–instruction loop that selects and executes actions, not the model alone or an independently accountable person. A *skill* is a reusable procedural resource describing when and how to act and verify the outcome, distinct from an execution history or a model-weight update. This procedural distinction follows Jiang et al. [47, Sec. 1]; the pilot's handover note is retained context, not a separately manipulated skill treatment.

Working behaviour means completing an intended task through the running system, including the identity, data and service dependencies needed for that task. A healthy endpoint or a passing repository test alone does not establish it. *Decay* denotes loss of previously working behaviour or increasing difficulty of understanding and changing a system. The case records lost usability during replacement; any decline in intrinsic code quality remains unmeasured. Deployment drift could produce the same observation. *Adaptability* is operationalised prospectively as success and cost on the fixed-budget, unforeseen change request, not as a universal measure of maintainability. *Consolidation* means scheduled structural work intended to preserve observable behaviour, with tests checking that intention; passing finite tests cannot prove semantic equivalence.

Five measurement categories remain separate. PRs, lines and words describe *production volume*; checks, reviews and receipts describe *validation activity*; observed user tasks describe *system behaviour*; code metrics and the change request probe *internal quality and changeability*; and corrective help requested or supplied describes *intervention*. Only the first three have case observations. The case has no longitudinal complexity measure, controlled changeability task or systematically coded intervention dataset. Neither volume nor validation activity is used as a direct estimate of decay.

3.6 Chain of evidence and reflexivity

Table 1 separates observations, interpretations and propositions. The table organises the evidence already presented; it adds no newly coded observations. Repository sources refer to the pinned commit unless identified as author reports. Log entries before 25 September are retrospective reconstructions; later dated entries are the project's contemporaneous record, not independent corroboration of every statement they contain. The 22 and 24 October instructions, historical model use and intermittent product use remain author reports where original records have not been recovered.

The author selected the episodes, designed the coordinator and had an interest in successful delivery. Those roles make a coherent retrospective explanation attractive. Developers' experience-based beliefs need not correspond with project evidence [29]; that methodological caution applies to the author's account as well as agent-generated claims. AI-written records are treated as accounts of project activity, with AI also assisting synthesis. Neither provides independent corroboration. No independent second coding or empirical peer-debrief pass has been completed. The supplied editorial critique prompted stronger boundaries and explicit rival explanations, but does not fill that gap. An independent check of O1–O5 against retained source and execution records remains necessary before presenting the interpretations as independently corroborated. Jiang et al. make mechanism analysis inspectable through matched trajectories, evidence-linked labels and a separate human check of taxonomy construction, while distinguishing that check from exhaustive human coding [47, Sec. 4, App. A.2]. Here the five mechanisms organise selected episodes; they are not frequencies derived from such a coding pipeline or effects isolated by matched executions. The machine-readable index anc/case-evidence-matrix.json separates reported locators, author accounts and missing execution or release identifiers. Each null identifier marks an evidence link that still needs checking.

Table 1: Observation–interpretation–proposition audit trail. Interpretations are contestable; the proposed tests have not been performed.

ID	Observation and provenance	Interpretation	Proposition or discriminating evidence
O1	Log, 8–14 Oct, plus author report: an MVP with sign-in and realistic synthetic data was used; 16 Oct record: replacement has one case, no staff and no model.	Recorded engineering completion did not preserve the owner's usable workflow.	Independently execute the same journey before and after migration; separate configuration failure from code regression.
O2	GitHub metadata: 183 PRs merged, 15–24 Oct; scorer records 5/5 completeness; author could not sign in. Dated API/scorer captures and their release/scope correspondence remain unverified.	Recorded completion and usability differed in the chronology; a same-release discrepancy is not yet established.	Link score, capability scope, deployment/configuration and the failed journey before testing agreement.
O3	BRD/PRD versions and log: growth before and during the loop; PR #445: two changed lines and a 78-line evidence record.	Requirements and process records may consume attention without delivering equivalent feedback.	Compare review effort and validated behaviour, not word counts alone.
O4	Log reference tracing: last simplicity citation on 13 Oct; missing registration/staff bindings in the replacement.	Some lessons may have fallen off the effective read path. Absence of a citation is not proof of absence of knowledge.	Inspect actual session inputs and test whether the prior lesson was available, understood and executable.
O5	Coordinator definition and ADR 0006: committed RPI records, full gates, delegated merging and migration priority; author still resolved blockers.	Reduced product-use feedback coexisted with substantial oversight.	Distinguish outcome-level feedback from approval activity; test the consolidation package separately from the migration.

4 Exploratory testbed: observations

Table 2 summarises the case and Table 3 compares the periods before and during the delivery loop.

Table 2: Timeline of the case (all dates 2026).

Dates	Phase	What happened
11 Sep – 7 Oct	Research and requirements	Sixteen regulatory sources converted; role agents reject five end-to-end agents (26 Sep); BRD and PRD drafted and repeatedly revised; prototypes reviewed; participant usability feedback recorded (7 Oct).
8 – 14 Oct	A usable MVP	MVP built and reshaped through several feedback passes in one day (8 Oct); running on Azure with SQL, blob storage and two replicas over 980 synthetic cases (9 Oct); rebuilt with Entra ID sign-in, used by the participant (12–13 Oct); five assistant paths exercised against a real model (14 Oct).
14 – 15 Oct	Backlog and coordinator	Backlog generated as GitHub issues; coordinator created and hardened within a day; an ADR makes migrating the MVP to a new source layout the first priority.
15 – 22 Oct	The loop at scale	Pull requests merged at about eighteen a day; the log grows by about 37,000 words; the migrated deployment holds one fictional case, no staff and no model deployment (16 Oct); browser sign-in blocked; validation moved off hosted CI to save budget (18 Oct); cloud cost ceiling raised (22 Oct); new cross-cutting requirements appended to open issues (22 Oct).
22 – 24 Oct	Asking for something that runs	Ten issues labelled *feedback-first*; instruction to make employee and resident sites available as soon as possible with at most eight parallel items; checkpoint and refactoring instruction (24 Oct).

Table 3: Before and during the delivery loop. Counts from the repository at commit `29c6b51` and the GitHub API on 24 October 2026; hedging phrases as defined in Appendix C.

Measure	Before the loop	During the loop
Pull requests merged	52 (26 Sep – 14 Oct; 2.7 a day)	183 (15 – 24 Oct; 18.3 a day)
Knowledge-base log entries	126 (to 14 Oct)	242 (15 – 22 Oct)
Words appended to the log	19,967 to 14 Oct (about 1,000 a day from 25 Sep)	36,755 (15 – 22 Oct; about 4,600 a day)
Hedging phrases per 1,000 log words	1.2	3.8
Log entries citing the simplicity requirements	4 (last on 13 Oct)	0
Participant-usable deployment	Entra sign-in, 980 synthetic cases, live model calls (12 – 14 Oct)	One fictional case, no staff, no model deployment (16 Oct); sign-in blocked

4.1 Research, requirements and a usable MVP

Table 4 adds code-level activity from the pinned history. The script classifies paths and compares each selected first-parent commit with its first parent, detecting renames. The integration view includes multi-parent merges and PR-numbered squash commits: 31 before the loop and 177 during it. These are not the API pull-request counts in Table 3; the literal-merge subset, seven in each period, is also reported in `anc/repo_history_metrics.out.txt`. Classification rules and commands are printed there. These figures quantify editing activity. Net code size, maintainability and delivered value require different measures. No complexity comparison was made because `lizard` was not installed.

Table 4: Text lines added and removed in first-parent integrations, including PR-numbered squash commits, through `29c6b51`. Detected pure renames add no lines; binary deltas are excluded.

Path class	8–14 Oct		15–24 Oct	
	Added	Removed	Added	Removed
Product source	8,136	421	23,408	2,184
Tests	6,871	537	73,478	1,416
Documentation	12,065	1,297	25,210	2,455
Tracking artefacts	238,185	2,642	240,369	1
Infrastructure	2,708	73	6,162	485
CI and workflows	240	64	897	246
Other	25,320	1,099	72,711	1,776

The September work was research and requirements preparation, not application building or a project-free baseline. The agents assembled a regulatory corpus, sector baselines and a modelled customer profile, and the role agents produced a genuinely useful early verdict: each requested *end-to-end* agent bundled deterministic controls, accountable human decisions and a smaller set of tasks suited to bounded AI, so none should be built as such. Requirements were written, reviewed by an automated quality reviewer and revised many times. Application prototypes followed in October.

On 7 October the author used the latest prototype and recorded usability problems. The next day the agents combined those notes with critiques of three prototypes by the UX, Design Thinking and role agents into ten simplicity requirements (SA-001 to SA-010): a default view listing only the user's cases, each with one plain status and one next action; a case that opens full screen; blockers explained in plain words with an owner; no role simulator in user views; optional AI that never blocks manual work; and so on. On 8 October they built an MVP that followed those requirements and then reshaped it through several rounds of participant feedback the same day: filters, an overview map, visits, handover suggestions and document uploads. On 9 October it ran on Azure with Azure SQL, blob storage and two replicas over 980 synthetic cases. On 12 October the stack was rebuilt from an empty resource group with Entra ID sign-in; the author reports signing in and using it. A missing logo and an account switch that only signed the user out were fixed within a day. On 14 October five assistant paths produced cited proposals through a real model deployment, using 6,884 tokens across five calls. The author tested an initial proof of concept for a couple of days, discarded it and used the MVP for a few days to refine ideas and the user experience. This was intermittent use, not daily use throughout the week. Those ideas informed the requirements, ADRs and hierarchical backlog.

The period 8–14 October supplies a within-case contrast with 15–24 October: intermittent use of a usable MVP versus no realistic end-to-end task completed in the replacement during the autonomous loop. The author continued handling approvals and technical blockers, but could no longer complete realistic tasks in the replacement. This contrast is consistent with the importance of feedback, but cannot establish that removing product-use feedback caused the

outcome: the migration, workload, gate rules, models and deployment conditions also changed, and the periods were not assigned at random.

4.2 Bounded within-case contrasts

Table 5 compares episodes and negative cases already in the chronology. It is an analytical contrast, not a new PR-cohort study or an exhaustive search for disconfirming cases. In particular, documentation expansion preceded autonomy and the tracking/source churn ratio fell during the loop. Both limit a story in which autonomous coordination alone created the problem. The successful health and restart checks are also real successes; they did not exercise the same outcome as the blocked sign-in journey.

Table 5: Within-case contrasts and their limits, using the existing chronology and measurements.

Contrast	Recorded difference or negative case	What it can and cannot discriminate
Product use, 8–14 versus 15–24 Oct	Intermittent MVP use and same-day feedback versus no completed realistic replacement journey.	Consistent with feedback starvation; migration, environment, workload and models changed at the same time.
Requirements before versus during the loop	Growth and review overhead already existed before autonomous coordination.	Weakens an autonomy-onset explanation; there is no matched small-specification comparison.
Integration artefacts, 8–14 versus 15–24 Oct	Tracking/product-source additions fell from 29.28 to 10.27 while absolute output remained high.	Volume is not a monotonic decay measure; churn does not assess understanding or usability.
Replacement engineering checks versus user task	Health, restart and unauthenticated-request rejection passed; real-user sign-in remained blocked.	Shows the boundary of those checks, not that passing checks were fabricated or that intrinsic code quality declined.
Domain challenge versus backlog execution	Early role agents rejected the five-agent proposal; later sessions delivered the specified issues.	Negative case against inevitable agent compliance; role, task and human interaction differ, so this is not a treatment comparison.

4.3 The backlog and the coordinator

Late on 14 October the backlog manager generated the backlog from the BRD, PRD and ADRs. Many issues carried a label stating that part of the work already existed in the MVP. On 15 October the author created the coordinator and, as Section 3.3 describes, hardened it within the day. The log recorded 91 entries that day, against at most 19 on any earlier day. The accepted source-layout ADR made the migration of the MVP's code the first priority; the MVP itself was to be retired.

4.4 The loop at scale

From 15 to 24 October, 183 pull requests were merged, compared with 52 in the nineteen days before. The knowledge-base log grew by 36,755 words between 15 and 22 October, 31,005 of them in the first three days (Figure 4). The density of lexical hedge matches, such as "remains unmet", "not inferred" or "not accepted", rose from 1.2 to 3.8 per thousand words.

The migration succeeded on its own terms. A private deployment built from the new source layout passed health checks, rejected unauthenticated requests and survived restarts. But it held one fictional case, no staff accounts and no model deployment, against the 980 cases, eleven staff identities and five working assistants of the system it replaced. A public entry point was restored through an additional edge service, yet real-user sign-in stayed blocked because the new deployment had neither an application registration nor staff bindings, the very things set up and verified on 12 October. Recovering them became a chain of issues, each with its own research,

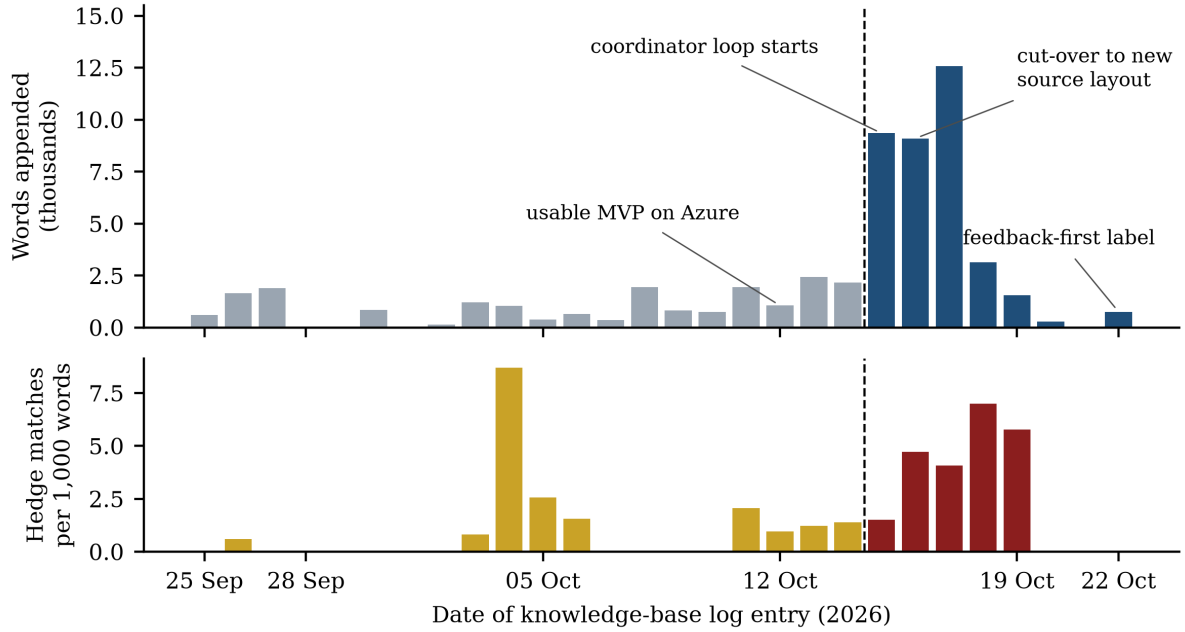


Figure 4: Words appended to the knowledge-base log per day (top) and lexical hedge matches per 1,000 words that day (bottom), from the packaged daily data. Days with no logged words have undefined density and no lower bar. The dashed line marks the coordinator's start on 15 October 2026.

plans, reviews and corrections. To preserve the hosted CI budget, validation moved to a local gate on 18 October (issue #413). The demonstrator's cloud resources accrued USD 189.94 between 1 and 20 October, with a forecast above the USD 500 owner-set ceiling; on 22 October the author raised it to USD 700, deleted a test edge profile and scaled a demo service to zero. The same day, after a mentor raised the economics of AI consumption, the agents produced new BRD and PRD versions and appended a new set of acceptance criteria to fifteen open issues.

A Copilot usage dashboard captured on 24 October showed USD 1,809 of price-list consumption value, 10,229,625,825 tokens and 100,938 requests in the rolling last 28 days. These were account-wide totals across all work, not AMA-only measurements or an invoice. The token total included 10,081,179,684 cached tokens. Surface and model breakdowns described the same total and must not be added together; the requests were not established as premium requests. The screenshots do not isolate 15–24 October, establish exact window boundaries or support an estimate of the loop's incremental cost.

4.5 Asking for something that runs

On 22 October the author requested a usable increment. A *feedback-first* label, described as prioritising "the bounded usable synthetic demo slice", was applied to the relevant issues, ten of them by 24 October. On 24 October the author requested employee and resident sites to be available as soon as possible, with at most eight items in parallel. These instruction dates and the eight-item cap are author-confirmed recollections, not quotations recovered from the pinned log. The agents created the smallest independently runnable resident preview they could design (issue #444): read-only, synthetic and bound to the local machine, with thirteen acceptance criteria. The same day a two-line fix to the rubric scorer arrived with a 78-line evidence record and an explicit, one-time owner exception to interrupt the full local gate (pull request #445). Following Beck's advice to alternate features with consolidation, the author also instructed the coordinator to stop at checkpoints, review what had been built, exercise the functionality and refactor. The outcome of that instruction is outside the case's evaluation boundary.

5 Five candidate mechanisms

Five mechanisms organise the gap between a pull request merged roughly every eighty minutes and the absence of a usable replacement. Table 6 summarises them; Appendix B sets out a definition, indicators, counter-evidence, rivals and testable implication for each. Together they connect the inherited-state problem to the way this delivery loop selected work, carried lessons and checked outcomes.

Table 6: Five candidate mechanisms, case indicators, relevant prior work and proposed early-warning signals. Indicators are not causal tests.

Candidate	Case indicators	Related prior work	Early-warning signal
Specification inflation	BRD ≈37,900 and PRD ≈33,900 words; 192 issues; 17 criteria on one issue and 13 on the smallest preview	Royce; Gall; Brooks; Böckeler; Beck	Specification growth versus human-validated behaviour
Proxy capture	Artefacts made a merge gate; 78-line evidence record for a two-line fix; score 5/5 while sign-in is blocked (release/scope linkage pending); hedge density tripled	Goodhart; Strathern; Beck's genie; ImpossibleBench; METR	Artefacts per behaviour change; scores that disagree with a live demo
Learning evaporation	Lessons not carried in the described briefs; last cited 13 Oct; read path ≈139,000 words	Naur; context studies; multi-agent failure taxonomy	Lessons that exist only as prose; problems solved twice
Process-gate inflation	Full gate on every push; CI minutes rationed; exception for a two-line fix; 18 merges a day delegated	Bainbridge; ironies of generative AI; Murphy; Goldratt	Gate time per change; review minutes per merge
Feedback starvation	Working MVP frozen; replacement unusable after ten days; request for a demo turned into more issues	XP; incremental delivery; tracer bullets; Agile Manifesto; DORA	Time to first human use; age of last usable increment

They occupy different points in the delivery process (Table 7). Specification growth increases the obligations and interactions that later work must preserve. Learning evaporation concerns the path by which a relevant lesson reaches the next session. Proxy capture concerns what selects and approves work; process-gate inflation and feedback starvation concern the capacity to detect and correct cumulative problems. None needs to be called error propagation to explain how an inherited fault can survive.

Until 24 October the loop had no scheduled cumulative consolidation step. Its large restructuring, the source-layout migration, was planned as backlog items; local refactoring could still occur within individual issues. The pilot therefore separates retained regression burden (H2), descriptive structural trajectories and later changeability (H6). Formal fault-lineage annotation would address the stronger question of whether an earlier fault generates distinct later failures; it is deferred rather than added to the pilot's workload.

5.1 Specification inflation

Interpretation: specification growth outpaced the behaviour validated through product use. By 22 October the BRD had reached version 0.19 and about 37,900 words, with requirement identifiers up to FR-052 and AC-103; the PRD had reached version 0.18 and about 33,900 words, with identifiers up to FR-061 and AC-110. The backlog inherited that scale and grew to 192 issues. Cross-cutting concerns were appended to existing issues rather than sequenced as their own work: a responsible-AI section on 15 October and an AI-consumption section on 22 October. Issue #126, on pilot release gates, ended with seventeen acceptance criteria in three sections. Even the request for the smallest runnable preview arrived with thirteen.

Table 7: Accumulation, transmission and correction in the working model.

Process	Question	In this case
Accumulation	What obligations and change burden does each feature leave behind? [54, 55]	No scheduled cumulative consolidation until 24 Oct; restructuring planned as backlog items
Transmission	Which implementations, rules and lessons shape the next change? [74, 86]	Growing specifications and read path; previously working identity setup not preserved
Selection and correction	Do review and tests expose the cumulative consequences before more work depends on them? [26, 94]	Proxy capture; process-gate inflation; feedback starvation

This expansion began before the autonomous backlog loop, including after participant prototype and MVP feedback. The requirements agents sought fully specified BRDs and PRDs, named approvers and resolved ownership before building. The author requested explicit demonstrator assumptions and imperfect increments that could produce feedback. Smaller, iterative requirements documents are a plausible alternative, not an intervention tested in this case.

The pattern resembles Böckeler's small bug that became sixteen acceptance criteria [19], at the scale of a programme, and the risks of late integration described by Royce and Gall [35, 80]. Brooks locates essential difficulty in the conceptual construct rather than its representation [21]; repeating a requirement in more places need not resolve its ambiguity. The benchmark evidence points the same way: agents given only a requirements document lose global coherence [30]. Beck's metaphor fits: every feature, and every acceptance criterion, burns options [13, 15]. The early-warning signal is the ratio of specification growth to behaviour a human has validated in the same period. No replacement journey was reported complete during the loop, but a systematic count of validated behaviours was not collected; this ratio cannot be estimated from the document counts alone.

5.2 Proxy capture

In this case, artefacts made into gates appeared to displace outcomes. Three patterns motivate this interpretation.

Merged pull requests are a weak proxy for delivered value. Murphy-Hill et al. report roughly 24% more merges among adopters in Microsoft's rollout, while explicitly distinguishing merges from value [67]. Peralta et al. studied 9,799 human-reviewed agent pull requests; in their manually inspected samples, only 35.7% of rejections reflected clear agent failures and 15.4% of merges required explicit reviewer involvement [77]. AIDev shows the scale of this measurement problem: 932,791 agent-authored pull requests from five agents across 116,211 repositories, not a count of delivered outcomes [56].

Process. When the first waves of sessions skipped the research and planning phases, the author made the presence of their artefacts a merge condition. From then on every pull request carried research, plan, change and review records; the artefacts were guaranteed, the thinking they were meant to evidence was not. Pull request #445 changed two lines of behaviour and added a 78-line evidence record that reports its own artefact completion as 100%. HVE Core treats these files as git-ignored working memory; the added gate turned them into the deliverable.

In the first-parent integration view, tracking-artefact additions were 10.27 times product-source additions during the loop, compared with 29.28 before it (anc/repo_history_metrics.out .txt). Artefact volume remained high during the loop, while its ratio to product-source additions fell. Churn alone does not establish code decay.

Evaluation. The rubric scorer, a language model reading the repository, rated implemen-

tation completeness at five out of five, "all required capabilities working end to end", citing a README and a check script, at a time when the author could not sign in to the replacement. The score and blocked sign-in occurred during the same period. It remains necessary to check whether the score covered that release and journey (Section 3.6). The judge read repository text rather than executing the intended journey. Neither its repeatability nor its agreement with observed product behaviour was measured; its score cannot substitute for that behavioural check.

Evidence boundaries. The author had instructed the agents never to fabricate approvals or overstate evidence. The record contains explicit statements of unmet acceptance, and lexical hedge density more than tripled. These observations do not establish exhaustive compliance or measure honesty. Issue #349 records that a browser redirect "is not successful sign-in". The agents were right, and still nobody could sign in. Honest boundary statements became the output.

Goodhart and Strathern predict the first two levels [40, 87]; Beck observed agents deleting tests to finish [72], and ImpossibleBench measured it [96]. The METR study adds the human side: the feeling of progress is an unreliable instrument [16]. The early-warning signals are the ratio of evidence artefacts to behaviour changes, the share of merged pull requests that change what a user can do, hedge density, and any divergence between automated scores and a live demo.

5.3 Learning evaporation

Some recorded lessons appeared to fall off the loop's read path, while important design decisions survived. Maintaining the structured log and memory became difficult as the record grew. Important design decisions survived, even as some recorded lessons appeared to be lost. Before the loop, participant use of the MVP produced concrete lessons: the simplicity requirements, a deliberately public development profile for throwaway stacks, and the callback, role and staff-binding steps that made sign-in work. All were recorded, in the PRD, in a simplicity review and in infrastructure runbooks. The coordinator's briefs carried each issue's text, acceptance criteria and the repository conventions, but not those lessons. The log cites the simplicity requirements for the last time on 13 October; the development profile appears only on 8 October. The migrated deployment lacked the registration and staff bindings established on 12 October, and sign-in was blocked.

The read path grew. The knowledge-base index, which every agent is told to read first, accumulated 1,069 words of issue-specific status notes ahead of its first instruction. The index, the log, the BRD and the PRD, the documents that define the project's state and intent, total about 139,000 words, or roughly 180,000 tokens at a typical ratio of 1.3 tokens per word. Long contexts can reduce reliable retrieval even when the material fits in the context window [43, 57]; the case does not measure any session's effective attention capacity.

Instruction debt and learning evaporation can therefore coexist. A correction may remain in an expanding agent definition while the evidence explaining when it applies falls off the next session's read path. The result can be a larger instruction set with weaker access to the relevant lesson. However, unavailable guidance, accessed-but-misapplied guidance and guidance insufficient for the task are different explanations [47, Secs. 5.2–5.5]. Without actual session inputs and access/use traces, this case cannot distinguish them.

Naur explains why recording was not enough. A program's theory lives in the people who build it, and a second team with full documentation can still destroy a design's simplicity [68]. In an agent loop, every session may face an analogous reconstruction problem. The comparison with Naur is interpretive. The case provides no measure of lost understanding. Advice to share full context between agents [92] and the failure modes catalogued for multi-agent systems [22] describe the same risk. HVE Core's choice to carry context in files on disk is necessary but not sufficient: the files must be on the read path, short enough to read and, ideally, executable. The

early-warning signals are lessons that exist only as prose, a read path that grows faster than the code, and problems that have to be solved twice.

Pavlenko et al.'s World Model offers a different design: preserve raw evidence, derive rebuildable views and retrieve compact, task-specific context with provenance and access controls. Updates are reviewed, and measured outcomes feed later decisions [76]. This distinguishes storing lessons from making them usable in the next decision; a small read path need not mean discarding history. This architecture offers another way to retrieve accumulated lessons. Its effect on learning evaporation in this case remains untested.

5.4 Process-gate inflation

Interpretation: repeated gates appeared disproportionately burdensome; comparative effort was not measured. Each new gate required execution and review. Within ten days the coordinator required, for every issue, the full RPI loop, a complete local gate before every push and after every correction or rebase, independent review, up to two correction cycles, board updates and evidence-based ticking of acceptance criteria. Hosted CI minutes had to be rationed, the full gate had to be interrupted by explicit exception for a two-line fix, and paid cloud validation stayed blocked even after the cost ceiling was raised.

The author perceived attention as more constrained than compute, but their relative scarcity was not measured. Reviewing eighteen merged pull requests a day in depth was judged impractical, and merging was delegated to the coordinator. The author still answered approvals and technical blockers, but stopped getting feedback from using a replacement product. RPI critiques and reviews addressed the current issue rather than the cumulative system; there was no regular phase for reflective consolidation or generalisation across completed issues. This is Bainbridge's irony in miniature [5]: the automation took the part of the work that supplied direct product-use feedback and left the participant monitoring a stream of plausible, well-documented changes. It matches the shift from producing to evaluating reported for generative AI [85], the delegation demands it creates [88]. Higher confidence in AI was associated with reduced self-reported critical scrutiny in the cited study [53], not established as its cause. Murphy's diagnosis is a plausible interpretation: throughput increased while the path to a usable outcome remained blocked [39, 66]. The bottleneck itself was not instrumented. Martin's argument for executable feedback from tests and code-analysis tools [60] motivates a proposed gate that checks product behaviour rather than inferring it from paperwork; the interview does not establish that such a gate would have prevented this failure. The early-warning signals are gate time per change, human review minutes per merged pull request, work in progress, and cost per merged pull request compared with cost per accepted outcome.

5.5 Feedback starvation

Candidate mechanism: insufficient independent, outcome-level feedback even though artefact-level validation is abundant. This differs from merely having few tests, a long feedback delay or missing context: many checks can pass while none disconfirms the claim that the intended user journey works. Outcome-level independence here means evidence obtained by exercising the outcome rather than rereading its completion record; it need not imply a different model. It does not establish independent human judgment. The loop prioritised completing the specified plan rather than shortening the path to such feedback. Feedback starvation is a central interpretive proposition, although the case cannot separate its contribution from the other mechanisms. Nothing in the coordinator kept a usable system alive: no invariant required the participant's sign-in and core workflow to remain usable after every merge. Completion was assessed through artefacts, criteria and gates. In the MVP week the feedback loop included intermittent participant use, with some changes made the same day. From 15 October the first priority was restructuring code, the working MVP was frozen, and its replacement had not become usable by

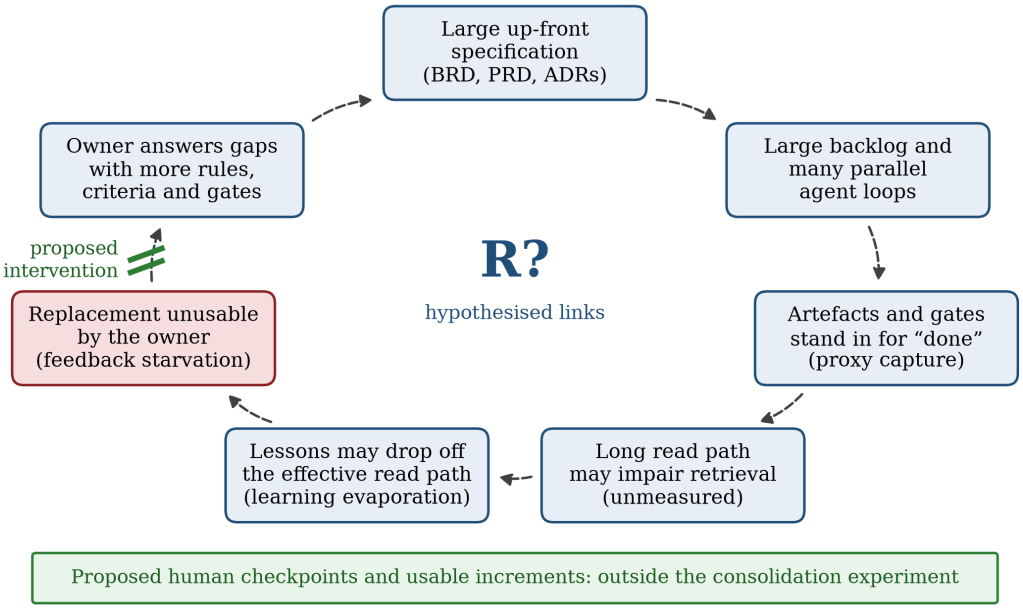


Figure 5: Proposed reinforcing relationships drawn from the case interpretation. Their causal effects remain unmeasured. Context overload is unmeasured. The proposed human-checkpoint intervention differs from the consolidation package tested in Section 7.

the time of the case cut-off. When a usable increment was requested, the request was absorbed by the same machinery: a label, new issues, thirteen acceptance criteria and the full set of gates.

Several older practices in Section 2.8 make feedback and working behaviour explicit: short releases and test-first development [8, 9], incremental delivery in Crystal Clear [25], tracer bullets [45], and working software over comprehensive documentation [2]. Beck's warning that fully agentic work without feedback goes off the rails [15], DORA's emphasis on small batches and user focus [31] and Murphy's list of real bottlenecks, which includes missing feedback [66], motivate this interpretation but do not independently diagnose the case. The early-warning signals are the time from the start of a cycle to the first human use of a running increment, and the age of the last increment a human has verified as usable. The case records no completed replacement journey during the ten-day loop; the exact last-use timestamp remains unverified.

5.6 A proposed reinforcing relationship

The mechanisms suggest a possible reinforcing loop (Figure 5). In this case a large specification accompanied a large backlog and parallel sessions. Artefact-based gates generated more documents; some recorded lessons were absent from later briefs; regressions were not exposed through routine product-use feedback. The author responded to gaps with more rules, criteria and gates. That sequence is consistent with reinforcement, but does not establish each causal link. A running increment and a human checkpoint are proposed conditions of every cycle, to test whether they interrupt it.

5.7 Case validity and rival explanations

The five categories are not independent causes identified by these data. The same missing identity and staff bindings can indicate deferred migration work, incomplete deployment acceptance or a lost lesson. Discriminating them requires the deployment comparison and actual session-input trace identified in Table 1, not another count of documents or citations. Similarly, a growing read path does not demonstrate context overflow, and an expensive gate is not neces-

sarily a disproportionate gate. Per-change gate time, avoided defects and human effort were not measured.

The source-layout migration, owner-selected priorities and delegated merging changed alongside coordination. Scope additions, budget constraints, model/tool changes and deployment conditions also differ between periods. These are direct rivals to an autonomy-only account. Tables 11 and 12 in Appendix A retain the full register, separating recorded circumstances from plausible but untested alternatives. Bird et al.'s study of co-installed applications on Windows machines supplies a methodological analogy for checking environmental influences on reliability [18]. It neither tests AI-agent configuration nor diagnoses this case's sign-in failure.

Construct validity is limited by whitespace counts, lexical indicators and selectively reviewed records. Appropriate caution can raise hedge density; missing references do not prove missing knowledge. As coordinator designer, participant and analyst, the author selected the episodes and may favour a coherent retrospective explanation. AI synthesis is not an independent observer. Public records support recomputation but not independent corroboration of every statement. The single fictional-domain capstone and purposively selected external examples do not establish prevalence or general causal effects. The proposed experiment has separate identification limits in Section 7.6.

6 Consolidation hypothesis and harness

The observed operational failure and the prospective hypothesis are distinct. Missing deployment identity and staff bindings are concrete obstacles to sign-in; structural code consolidation would not necessarily restore either. Recovery of that journey requires configuration and end-to-end verification. Consolidation is worth testing for a different reason: repeated feature work may make later changes harder, even when deployment dependencies are correct. The experiment probes that possibility through retained tests and CR1; it does not claim that consolidation would have prevented this episode.

The idea is already practised. OpenAI describes replacing weekly manual clean-ups with recurring background agents that open small refactoring pull requests, alongside a documentation-gardening agent [70]. Anthropic asks each session to leave a clean state and update a progress file [3]. Neither account reports a controlled comparison of feature-only delivery with consolidation between feature steps. Darwin's continuous optimisation is a further precedent for automated improvement, but its performance evaluations do not isolate that intervention or test an unforeseen change request [76] (Section 2.6). RefactorBench already evaluates refactoring agents [37]; the narrower hypothesis concerns retained behaviour and later adaptation during repeated feature additions.

AutoSaddler's publication abstract already treats harness improvement as offline learning from execution traces, combining failure diagnosis, structured patches and validation-based update selection [63]. Agent-led reflective correction is therefore not the novelty claimed here. The proposed intervention instead consolidates the retained repository between feature steps and measures retained contracts and a withheld change request.

The proposed consolidation harness tests a bounded part of reflective correction. The harness schedules each pause. The agent is not authorised to reject features or redirect the product. After every feature the loop stops, and an agent:

1. runs the tests and a metrics script that lists complexity hotspots, duplicated blocks and lint warnings;
2. reviews the last feature's diff and the hotspots against a short checklist (duplication, long or complex functions, unclear names, dead code, rules from one feature leaking into another, tests that check implementation rather than behaviour);
3. refactors in small, behaviour-preserving steps, each applied through `test && commit ||`

`revert` [10], so that every change either keeps all tests green or disappears;

4. may add tests that pin current behaviour, but may not change or delete the example tests that define the features;
5. rewrites a design note of at most 300 words that says where each rule lives and what debt remains, replacing the previous one rather than appending to it;
6. stops when a quality budget is met or its time budget runs out.

Only then may the next feature start. Appendix D gives the prompt. The intervention combines refactoring, additional tests and a design-oriented note; it is not a pure refactoring treatment.

The hypothesis draws on Beck's alternation of features and futures [13, 15] and *Tidy First?* [11]. It is not free, and may not work. Agent refactorings can be incorrect or superficial [37]; human studies also caution against equating a refactoring label with a quality gain. Kim et al. report developers' refactoring challenges and benefits [50]; Bavota et al. find that observed refactorings do not consistently target or remove code smells [7]. These findings motivate checking outcomes rather than assuming improvement. Test-commit-or-revert protects only behaviour represented by its tests, not every intended contract [59]. Section 7 specifies how the package will be evaluated.

7 Testing the harness: an experiment protocol

The study compares scheduled post-feature consolidation with feature-only delivery and read-only review, building on existing agent clean-up and optimisation work [76]. It tests whether an agent can perform a bounded part of reflective correction, rather than whether another agent can autonomously steer the whole development process. Lulla et al.'s registered-report protocol, which treats agent configurations as experimental arms, provides a model for specifying the comparison before observing results [58]. The author designed the experiment with HVE Core's Experiment Designer as a Minimum Viable Experiment [46], informed by empirical-study guidance [6, 91]. The choices below incorporate the author's decisions of 25 October 2026.

Study status and provenance. The planned S1 first pass has not started. It is a twelve-trajectory pilot/estimation study, four runs per arm, intended to assess harness feasibility, endpoint behaviour, variation and failure modes before a larger study. No experimental results are reported. The reported private-kit tag is not yet a verified or registered protocol. This prospective manuscript amendment simplifies the endpoint hierarchy without changing the arms, sample or resource ceilings; it does not rewrite that tag. Any earlier smoke, calibration or pre-registration attempts must retain their actual labels, timestamps and exposure history and remain separate from this pilot. Their existence, completion and data seen must be checked rather than inferred from the current manuscript. The earlier ancillary proposals are historical records. Launch remains blocked by Section 7.7; no registration, precision exercise, independent review or execution is claimed complete here. The related work reviewed here does not isolate consolidation steps interleaved between successive feature additions. Darwin evaluates optimisation against measurable objectives [76]; SlopCodeBench shows that quality instructions alone shift where agents start but not how fast they degrade [71]. The harness tests a scheduled structural intervention and subsequent changeability instead.

7.1 Protocol at a glance

Table 8 summarises the pilot. Its unchanged arms and budgets support package comparisons, not a pure-refactoring or equal-total-resource estimand. The prospective endpoint amendment still needs reconciliation with the actual runner and analysis snapshot.

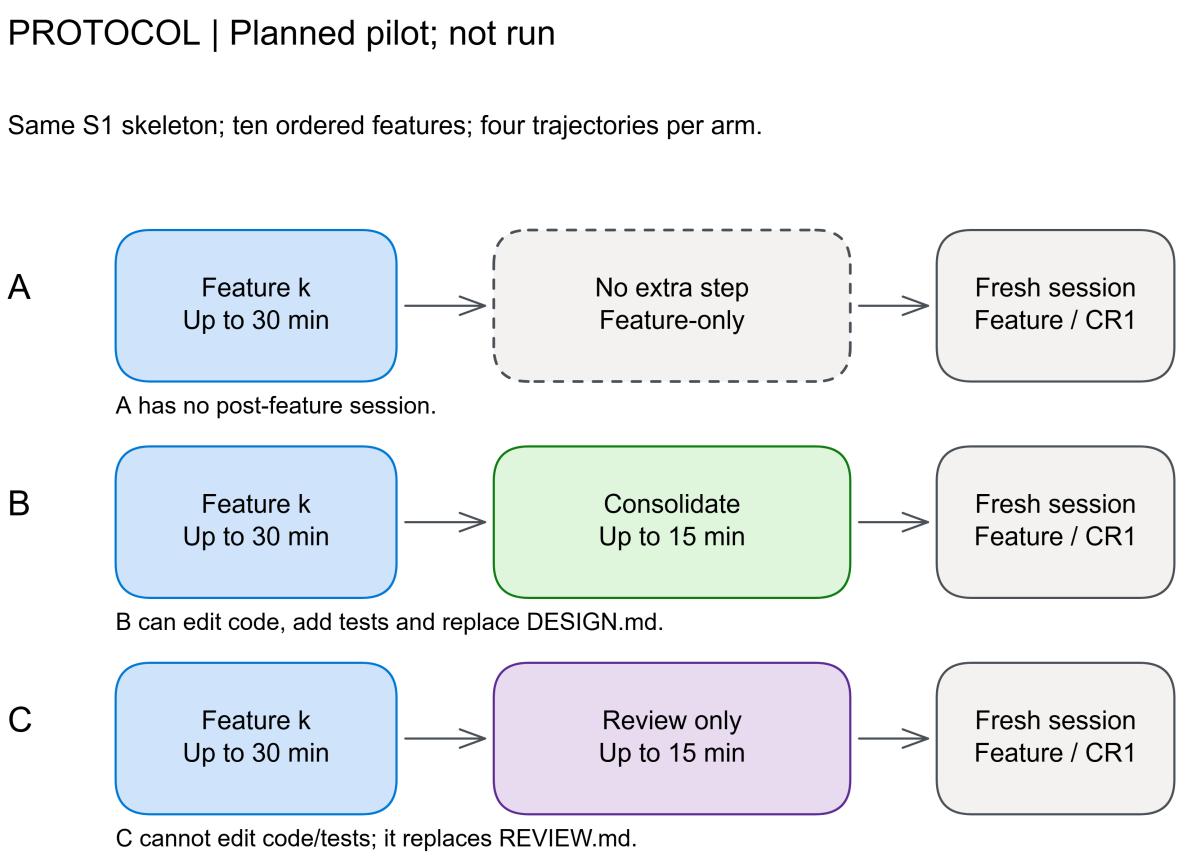


Figure 6: Prospective pilot design, not results. Each arm has four complete ten-feature trajectories and a final 45-minute change request (CR1). B and C add at most 15 minutes after each 30-minute feature step; A does not. B–A includes extra resources, while B–C also differs in code/test editing and handover content. Planned external evaluation does not establish access isolation; hidden-output visibility and failed-feature continuation must be resolved before launch.

Table 8: Summary of the twelve-trajectory, registration-pending pilot. Planned safeguards are not completed checks.

Item	Design and interpretation boundary
Study unit	One complete S1 repository–feature-sequence trajectory; four independent runs per arm. Feature steps and CR1 within a run are not new independent replicates.
Allocation	Planned randomised interleaving of arm runs on the same machine; one fixed ten-feature suite, not random assignment of repositories or scenarios.
B–A estimand	Effect of adding the complete consolidation package at equal feature-step budgets, with up to 50% extra agent time. Not an equal-total-budget or refactoring-only effect.
B–C estimand	Complete consolidation versus read-only review at equal post-feature time ceilings. Editing permission, added tests and DESIGN versus REVIEW notes also differ.
Endpoint hierarchy	Practical: final cumulative behavioural coverage (H1). Decay-specific: losses of previously passing contracts (H2). Future changeability: CR1 success and cost (H6). Resource cost is the practical counterbalance; other metrics are secondary descriptive outcomes.
Changeability	CR1 success and cost under 45 minutes; success also preserves every feature test passing before CR1. One fixed request is a task-specific probe, not a general maintainability measure.
Budgets	30 minutes per feature; 15 per B/C post-feature step; 45 for CR1. Credits and monetary cost remain distinct; final calibrated caps need checking.
Analysis	Pilot run-level estimates and endpoint/measurement feasibility. No seven-hypothesis verdicts, mixed-model significance claims or completed power simulation. All run outcomes and non-estimable measures remain visible.
Integrity	Fresh sessions, fixed prompts/tests/configuration and checksum checks are planned. Local execution does not guarantee hidden-test or reference inaccessibility; independent suite/access checks remain pending.
Human help	Primary runs unassisted; HIR/IDR descriptive. Corrective help is a retained, labelled deviation, not grounds for a replacement run. Telemetry coverage and annotation reliability remain unverified.
Status	Not running; no results. Runner/analysis reconciliation, independent suite/access review and registration remain launch blockers. Calibration data cannot be pooled into the twelve pilot trajectories.

7.2 Pilot questions and endpoint hierarchy

The pilot concentrates on retained behaviour, future changeability and the resources they require. Table 9 retains H1, H2 and H6 as traceability labels from the earlier proposal, not confirmatory tests. Final coverage is the primary practical outcome; retained-contract loss is the primary decay outcome; CR1 is the principal future-changeability probe. High final coverage after late repair must not conceal recurrent losses.

Table 9: Core pilot questions. A is feature-only delivery, B adds the consolidation harness, C adds a read-only review of equal budget. Estimates describe the pilot; no right/wrong hypothesis verdict is assigned.

ID	Question	Outcome	Interpretation
H1	What working behaviour remains?	Final cumulative hidden pass rate; run-level B–A and B–C differences	Combines acquisition, retention and repair; not itself a decay measure
H2	Is previously working behaviour lost?	Feature/post-feature contract losses, eligible risk sets, recovery and recurrence	Separates retention from never-acquired features and successful later repair
H6	Can the final repository absorb CR1?	Success under the fixed budget, preserving pre-CR1 passing tests; all-attempt cost	A single unforeseen change probe, not general or longitudinal maintainability

Resource cost is reported alongside all three outcomes, with AI credits as the primary consumption measure. Structural metrics, code size, comprehension and HIR/IDR are secondary descriptive analyses, not additional success tests. The former H3/H5 slope and cost rules are not pilot verdicts. H4's joint size/accuracy/time claim and H7's dependent half-gap rule are removed from the formal first-pass hypothesis set: their endpoint and non-inferiority requirements are not defined. No replacement margin or half-gap threshold is invented. Earlier thresholds remain historical proposal material, not seven pass/fail criteria for these twelve runs.

7.3 Design

Arms. Arm A delivers features one after another. Arm B runs the consolidation harness after every feature, with half the time budget of a feature step. B therefore has up to 50% more agent time per feature than A; the comparison is not time-matched. Resource use is compared directly. Arm C holds that extra time ceiling constant without refactoring and replaces consolidation with a read-only review of equal budget, to compare the complete consolidation package with review-only. B may add behaviour-pinning tests and writes an architectural `DESIGN.md`; C cannot change tests and writes issue-oriented `REVIEW.md`. Subsequent sessions read those different notes, so this comparison cannot isolate refactoring. Everything outside the post-feature intervention is identical: model and version, agent runtime, feature prompts, starting skeleton, tools and per-feature budgets.

Estimands and control limits. B–A estimates the incremental consolidation package at a fixed feature budget, including extra time and compute. B–C compares packages at equal post-feature time ceilings, not necessarily equal realised tokens, credits or elapsed time. C cannot repair detected defects, add tests or write B's design note. Any difference can therefore reflect permission to act, test additions, note content or their interactions, not structural consolidation alone. Review-only is not an active repair control. There is no equal-total-budget comparison, and this study cannot say what consolidation would displace under a fixed total budget. Those limitations constrain the causal interpretation even if allocation and the planned checks are implemented correctly.

The same pinned model receives a fresh session and a different brief; its conclusions still require verification. Its value must be demonstrated by behavioural and changeability outcomes, not inferred from the existence of another review role.

Runtime and budgets. The author selected `claude-sonnet-5.5`, medium reasoning effort and Copilot CLI 1.0.103, with automatic updates disabled. The wall-clock limits are 30 minutes per feature, 15 per consolidation or review, and 45 for CR1. The calibration-stage limit is 300 AI credits per session; any calibration-informed adjustment must be frozen and included in the registered configuration before the fresh study runs. The total calibration-and-pilot ceiling is 2,500 AI credits, raised from an initial 1,000-credit proposal. A separate $500 monetary budget was stated. Credit consumption and currency cost are not interchangeable without a verified billing conversion.

Suites. Suite S1 is a ten-feature retrofit case workbench drawn from the case domain, whose features deliberately interact: lifecycle, handoffs, returns, safety holds, resident consent, appointments with an escalation rule, role-based visibility, deterministic ranking and a hash-chained audit trail. The selected study uses all ten features and the full CR1, including the ranking and export prerequisites in F9–F10. Each feature has visible example tests and hidden acceptance tests, including interactions with earlier features. Tests and reference solutions are intended to remain outside the agent's working repository. S2 is a library-lending suite reserved for a separately authorised main study; benchmark adaptation is deferred.

Isolation. The planned pilot uses local execution rather than containers. The CLI access rules and separation of working directories provide a weaker boundary than container isolation. They do not establish that hidden tests, references or outside resources are unreachable. Residual access risk limits validity and must accompany any results; an absence of leak flags is not proof of isolation. CR1 is intended to remain unavailable to feature, consolidation and review sessions until its dedicated change-request session. The tagged packaging and actual non-exposure of its request text, as well as its tests, remain to be verified. Calling CR1 "unforeseen" depends on keeping it hidden until its dedicated session; that access boundary still needs verification.

Procedure and information flow. The procedure describes the intended design. Whether the private runner implements it remains to be verified. Repository context is passed between fresh sessions; the evaluator is outside the working repository. Access separation and result visibility still require verification.

1. A feature session receives its fixed prompt, visible example tests and the repository left by the previous step, including the arm-specific note. It may commit only when the visible tests pass.

2. If no green increment is committed, the runner resets it, removes that feature's example tests and records the feature as missing. Hidden tests for that feature remain in the cumulative denominator. Continuation when later features depend on it is an unresolved runner-policy decision.

3. The external evaluator records cumulative hidden-test results and code metrics after the step. Whether any score, failure message, test identifier or report becomes visible to subsequent agent sessions is unresolved; hidden test files being outside the repository does not settle this information boundary.

4. In B or C the assigned consolidation or review session precedes the next feature, with another external measurement afterwards. Whether it runs after a failed feature is unresolved. A has no post-feature session.

5. After the final scheduled post-feature step, preserve H1's end state; then start the dedicated CR1 session with the previously withheld request and its fixed budget. Score change success against the pre-CR1 passing tests.

All unresolved branches belong in the frozen runner and divergence rules before fresh runs, not in retrospective choices after inspecting outcomes. A separate budget-calibration stage is planned with one run per arm over ten features, without CR1; its data will not be pooled into the pilot. The selected pilot uses S1, arms A, B and C, with four independent runs per arm, ten

features and CR1. The twelve trajectories will assess feasibility and variation. Stable estimates for all seven original hypotheses would require a larger study. The pilot has 120 feature, 40 consolidation, 40 review and twelve change-request sessions: 212 sessions and an upper bound of 89 agent-hours. The separate calibration stage adds fifty sessions and twenty agent-hours, making 262 sessions and 109 agent-hours combined. These bounds exclude measurement and scheduling overhead and do not establish affordability under the credit ceiling. The sample and stopping rules must be versioned before launch; favourable or unfavourable pilot effects will not trigger extra runs. Later suites or increased replication need a new prospective plan informed by the pilot's variance and failure modes.

End state and change request. H1's end state is after the last feature's post-feature step (a no-op in A), before the change request. Change-request success requires all its hidden tests to pass and no feature test that passed at that end state to fail afterwards.

Endpoint interpretation. H1 combines feature acquisition, retention and repair. H2 follows previously passing contracts through loss and recovery; H6 probes a later change under the fixed budget. These outcomes must be read together. Better final coverage without fewer retained-contract losses supports improved implementation, not reduced decay.

Measures. Cumulative hidden pass rate, open regressions and how long they survive; mean cyclomatic complexity, the share of functions above complexity 10, duplicated lines and lint warnings per thousand lines; non-test lines, diff lines per step and documentation words generated; AI credits as the primary consumption measure, with tokens and elapsed time secondary; change-request success and cost; and, for S1, a proposed secondary comprehension assessment using five fixed questions about each final code base. Test integrity, through checksums of the example tests and counts of tests and assertions, is checked after every step. The planned code-metric settings are the runbook's isolated lint rules E, F, W, C90, B and SIM, duplication detection at fifty tokens and five lines, and complexity measured by lizard. Tool versions and settings must be fixed in the registration. Two independent comprehension reviewers are proposed; their agreement, scoring rules and effective blinding remain pending. This assessment will be descriptive, with no accuracy non-inferiority or twenty-percent time-saving verdict. Code size and structural metrics must be interpreted alongside functional coverage: an incomplete implementation can be small without being better. For H2, a newly introduced failing test alone is not an observed loss of previously passing behaviour. Section 7.4 proposes regression eligibility, measurement points and reporting denominators for prospective reconciliation with the private kit.

Intervention measures in prior work. These are cross-domain measurement precedents from robotic imitation learning, not studies of coding agents. HG-DAgger distinguishes human takeover and return of control [49]; ThriftyDAgger separates intervention counts, human actions and supervisor switching costs [44]. They motivate separating incidence and effort, not a software-decay claim or coding-harness effectiveness. Here HIR is cycle-level incidence, not events per hour or a human-control time share. Failed cycles remain included, and event counts and active human minutes are separate measures; fewer events need not mean less human work or monitoring burden.

Human intervention rate (HIR). As a secondary measure of harness quality, observed HIR records the proportion of attempted feature cycles receiving unplanned corrective human help. A cycle includes the feature and its assigned post-feature step, so B's additional sessions do not enlarge its denominator. Failed cycles remain included; repeated interventions in a cycle count once for the rate, with event counts recorded separately. For run r, let n_r be the number of attempted feature cycles and h_{rj} the number of qualifying corrective-help events in cycle j. Then

$$\mathrm{HIR}_r = \frac{N_{\mathrm{assisted},r}}{n_r} = \frac{1}{n_r}\sum_{j=1}^{n_r} \mathbf{1}\{h_{rj} > 0\}. \tag{1}$$

The indicator is one when the condition holds and zero otherwise; multiplying Equation 1 by 100

gives a percentage. By contrast, $\sum_j h_{rj}/n_r$ is events per cycle and can exceed one. Event counts and active human minutes are reported separately so that a brief clarification and a long repair do not appear equivalent merely because both affect one cycle. These are incidence and effort measures, not interchangeable estimates of total supervisory workload. If no cycle is attempted, the rate is undefined, not zero.

Primary runs receive no task-specific human coaching or code/test repair. The protocol therefore also records an intervention-demand rate with the same denominator: cycles containing an explicit request for such help or a block requiring manual corrective action. With $d_{rj} = 1$ for such a cycle and zero otherwise,

$$\mathrm{IDR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} d_{rj}. \tag{2}$$

A request need not result in assistance, and an unsolicited human correction need not follow a recorded request; the two are coded separately. Zero observed HIR under this policy is not evidence of autonomy. A timeout or automatic reset alone does not establish demand.

The intended sources are the session telemetry and retained logs. The proposed procedure uses Cowork to extract requests, blocks and actual interventions, followed by author checking against source records and adjudication of ambiguous cases. The extraction must link each event to its cycle, phase, reason, action and active human minutes, using rules fixed before fresh runs. Telemetry coverage of human-help requests and corrective actions, including actions outside the CLI, has not yet been verified. Missing records cannot be interpreted as zero interventions; incomplete coverage makes the affected measure not evaluable. Routine launches, planned evaluation, required approvals and safety stops are recorded separately, not treated as corrective rescue. Any corrective help supplied is a protocol deviation; the run is retained, labelled assisted and reported separately from unassisted estimates. Such deviations do not trigger replacement runs and may leave the unassisted comparison not evaluable at its predeclared sample size. Change-request interventions are reported separately, alongside completion, missing features, early termination and unattempted cycles. Lower HIR alone is not better: it must not reward silent failure or discourage necessary human judgement.

Author checking of an automated extraction is not independent annotation. No inter-rater agreement, stable severity coding or construct-validation result is available; these rates remain descriptive and their annotation reliability must not be inferred from a complete-looking event table.

7.4 Measuring decay prospectively

Proposed measurement amendment. The panel below makes H2's retention-specific outcome distinct from H1's final coverage and the secondary structural metrics. Its eligibility map and checkpoint rules must be reconciled with the actual runner and analysis plan before launch.

Follow previously working contracts, not only final scores. Figure 7 shows why final coverage alone cannot distinguish acquisition, retention and repair. Freeze a hidden contract panel and a feature-by-test eligibility map before execution. Distinguish invariants from behaviour intentionally changed by a later feature; expected changes need prospectively versioned oracles, not post-hoc removal of failing tests. Save the state immediately after feature t (F_t) and after its consolidation/review step (P_t); in A, $P_t = F_t$. Evaluate the same eligible contracts at both checkpoints and keep evaluator outputs outside agent context under the agreed information policy.

For run r, let $\mathcal{R}_{r,t}$ contain eligible unchanged contracts that passed at least one earlier end-of-cycle checkpoint P_u, $u < t$. Let $S^p_{r,t}(q)$ be the recorded binary result for contract q at phase

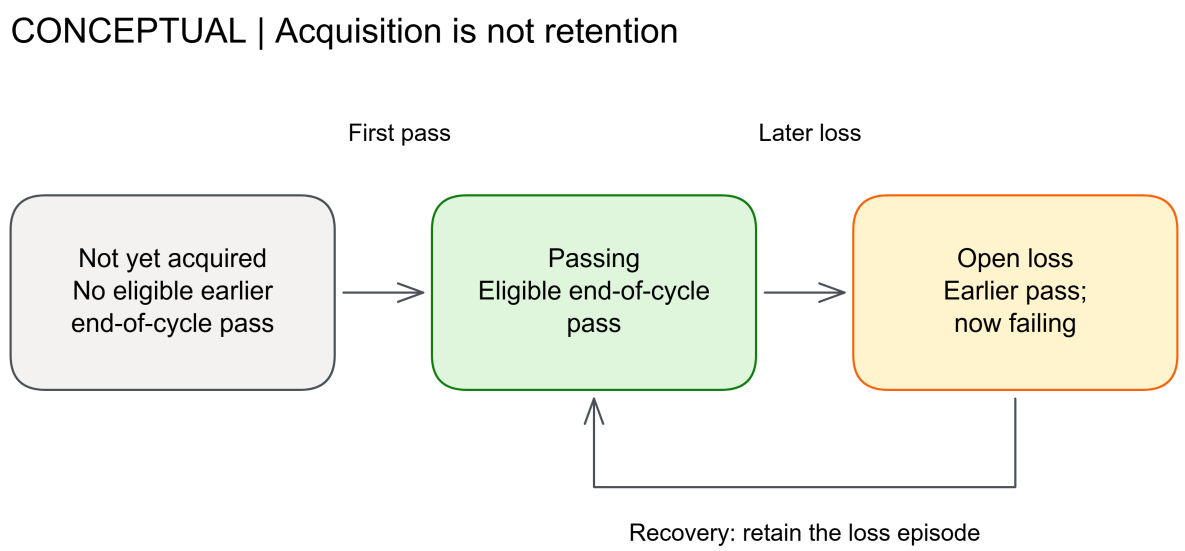


Figure 7: Conceptual contract states, not measured frequencies or example results. A never-passing eligible contract is an acquisition failure. An unchanged contract that passed an earlier end-of-cycle checkpoint can subsequently be lost and recovered. Recovery improves final coverage but does not erase a loss episode. The historical ever-passed risk set is distinct from the currently passing set; missing evaluations are unobserved, not failures.

$p \in \{F, P\}$. Proposed open-loss count and density are

$$L^p_{r,t} = \sum_{q \in \mathcal{R}_{r,t}} \mathbf{1}\{S^p_{r,t}(q) = 0\}, \qquad d^p_{r,t} = \frac{L^p_{r,t}}{|\mathcal{R}_{r,t}|}. \tag{3}$$

The density is undefined when the risk set is empty, not zero decay. A test that never passed is an acquisition failure, not a regression. Recovery removes an open loss but not its recorded episode: retain onset, recovery, recurrence and duration in feature cycles. Also distinguish a newly introduced failure from a loss still open from an earlier cycle. Report F_t and P_t separately so successful repair does not hide feature-step damage. Log rejected/reset attempts separately from losses in the retained repository state. Separately flag a newly delivered contract that passes F_t but fails P_t: this is same-cycle loss during the post-feature step, even though it is not yet in the inherited cohort. Unobserved results remain missing, not passes or failures; apply the prospectively fixed outage and missing-measurement policy.

The proposed H2 reading uses $L^P_{r,t}$ for the longitudinal open-regression series and $L^P_{r,10}$ for its final count comparison. Keep densities and accumulated test-cycle losses as descriptive companions, not substitute thresholds. Show the eligible, ever-passed, never-passed and currently passing counts alongside every loss series and H1. Eligibility depends on what an arm managed to acquire: fewer at-risk contracts must not make an incomplete arm look healthier. These are test-weighted measures; retain feature/requirement groupings so many assertions for one feature do not masquerade as many independent capabilities. Runs, not tests or checkpoints, remain the units for uncertainty. A zero A-arm loss count provides no denominator for a claimed relative reduction; report absolute counts and differences instead. An upward count can reflect more contracts at risk rather than greater decay risk, so count and density trajectories must be interpreted together.

Do not infer propagation from persistence. A repeated failure of the same contract is not a distinct propagated fault. Formal fault-lineage annotation and matched faulty/repaired-snapshot comparisons are deferred to the ancillary research agenda, not added to the pilot's required analyses.

Separate structural deterioration from legitimate growth. Keep the four structural metrics and frozen settings at every checkpoint, but also measure the independently checked cumulative reference implementation for each feature stage. Report raw trajectories and agent–reference gaps in the original metric units; do not combine them into an unvalidated decay score. Compare only compatible feature coverage, disclose omitted or missing features and retain all run outcomes. More code or complexity required by later features is not itself decay. Raw trajectories and reference gaps are descriptive sensitivity analyses, not the earlier H3 slope-pattern test. If the reference or metric checks fail, label the affected interpretation unavailable.

Distinguish changeability from changeability decay. Final CR1 success and cost compare the arms' final repositories; one final task cannot establish longitudinal changeability decay. Early/late probes and additional change requests are deferred to the ancillary research agenda and are outside the 212-session pilot.

Make the hypothesis capable of failing. First repeat oracle checks on unchanged snapshots and reference versions to expose flakiness or environment faults, using fixed repeat and failure rules. Within a trajectory, the paired checkpoints locate when losses appear and whether they are repaired; the arm contrasts test the consolidation package, not feature accumulation versus a randomised no-feature control. Stable retention and no structural excess would show no observed decay under this workload, not prove decay impossible. Better H1 without lower losses or structural deterioration supports improved implementation, not decay control. Growth matching the reference favours task complexity over deterioration; oracle instability leaves the affected decay measurement unresolved. Report these rival patterns, null outcomes and uncertainty rather than treating every final shortfall as support for the subtitle.

7.5 Analysis and bias controls

Report every trajectory's coverage, retention losses, CR1 outcome and consumption before arm summaries. Planned B–A and B–C comparisons use run-level differences in mean coverage and resource use, with the H2 checkpoint histories and their risk-set denominators shown separately. Code generation is nondeterministic [73]; repetition estimates variation but does not make feature steps independent. Structural metrics, comprehension and HIR/IDR use descriptive trajectories, counts and ranges. Mixed-effects significance tests are not a primary pilot analysis.

CR1 estimands. The proposed binary endpoint concerns all four allocated trajectories per arm. Agent-caused inability to reach or complete CR1 is unsuccessful, not a removable expensive failure. An infrastructure-caused, unobserved outcome remains missing under the prospectively fixed event rules. For complete outcomes, report the success fraction and absolute B–A and B–C risk differences. The proposed conservative interval uses equal-tailed 97.5% Clopper–Pearson intervals $[l_X, u_X]$ for each arm's success probability, giving $[l_B - u_A, u_B - l_A]$ for B–A and the analogous B–C interval. The two component intervals give at least 95% joint coverage under the binomial model; this does not supply family-wide coverage for all comparisons. Missing outcomes require explicit lower/upper outcome bounds, not a complete-case success rate. The implementation and boundary cases still need pre-run checks.

Resource estimands. Report actual AI credits through the final post-feature step and CR1 consumption separately, including failed, stopped and assisted sessions. Unused budget is not

expenditure. A CR1 cost of zero requires evidence that no session consumed credits; absent billing or telemetry is missing cost, not zero. All-attempt consumption is interpreted jointly with success, not as the cost of achieving a successful change. Successful-run CR1 cost is an explicitly conditional, secondary description; it is undefined in a zero-success arm. Do not select only successful runs for the primary cost comparison or interpret cheap failure as improvement. Credits, tokens, elapsed time and currency remain separate.

Show all run values and ranges alongside any pointwise, run-resampled intervals for coverage or cost. Their implementation, diagnostics and not-estimable rules must be frozen before launch; with four runs per arm, such intervals are feasibility outputs, not stable precision claims. The planned controls are randomised interleaving, fixed prompts and evaluator information boundaries, checksum checks, a divergence log and reporting every allocated run. Calibration data remain separate. No interval or observed effect triggers replacement or additional pilot runs.

7.6 Identification and precision limits

With four trajectories per arm, the 120 feature steps are repeated observations within twelve independent trajectories. They support pilot estimation and measurement checks, not seven stable effect claims. One suite, one proprietary model and one binary change request restrict transfer; a single final CR1 cannot establish general or longitudinal maintainability. No power or interval-coverage simulation has been completed. This sample is retained as pilot/estimation work, not justified by attainable p-values.

The pilot will inform a later precision exercise and replication decision, not be retrospectively promoted to a confirmatory study. Any later study needs its own pre-data contrast, endpoint hierarchy, multiplicity policy and simulation-based sample justification. Twelve clusters and pointwise intervals cannot supply those guarantees.

B–A includes extra resources; B–C also changes permission to edit, added tests and handover content. Neither identifies pure refactoring, and neither manipulates the five case mechanisms separately. The intended local access boundary, version pinning, suite/oracle stability and blinded review remain unverified. A provider service can change despite an exposed version pin [6]; note content can reveal treatment. Missing telemetry is not zero intervention. The specific launch requirements below must be resolved before these planned safeguards can be described as controls.

7.7 Pre-run decisions and status

The pilot is **not running; launch is blocked**. Table 10 lists unresolved decisions and unperformed checks. Each requires an author-approved, versioned resolution reconciled with the actual runner and analysis plan before launch. The manuscript alone does not freeze the private kit or establish independent acceptance.

Table 10: Outstanding launch blockers, not completed safeguards. Arms, sample and resource ceilings remain unchanged; the pilot endpoint amendment still requires runner/analysis reconciliation.

Boundary	Resolution and evidence required
Provenance	Record the private tag/SHA, checksums, pilot amendment, any earlier data seen, registration and authorised archive; preserve actual earlier labels.
Suite and access	Obtain independent suite/oracle review, test repeatability and actual hidden-test/reference/CR1 access probes before agent exposure.
Evaluator information	Freeze whether hidden scores, messages, identifiers or reports reach subsequent sessions; verify the runner enforces that policy.
Missing features	Specify B/C after a failed feature, prerequisite-dependent continuation, later recovery and test eligibility; retain missing features in the stated cumulative denominator.
Outages and deviations	Distinguish agent failure from infrastructure/service failure; fix retry, stopping, missing-measurement and assisted-run disposition.
Endpoint implementation	Reconcile H2 eligibility, feature/post-feature checkpoints and reference-gap analysis with the actual tagged runner. Check the proposed CR1 success/missingness and all-attempt cost estimands, interval boundary cases and any resampling diagnostics before registration.
Secondary scope	Decide before data collection whether comprehension will be collected; if so, freeze questions, scoring, timing, reviewer agreement and allocation/order. H4/H7 formal claims are deferred, not silently tested.
Precision	Record the twelve-cluster pilot purpose and interval limitations; no power simulation is claimed. A larger confirmatory study requires a separate simulation-based precision and replication plan.
Resources and versions	Verify calibrated credit caps against the separate currency budget; freeze exposed runtime, model, prompts and metric settings.
Human-help telemetry	Verify requests and outside-CLI actions are observable; freeze event extraction/adjudication; missing records are not zero help.

The pilot has not started. No first-pass results are reported. The launch blockers above and registration remain outstanding; future reporting must retain null, negative and inconclusive findings and distinguish any earlier calibration or smoke attempts from this study.

Planned results figures: awaiting data. The following figure briefs are prospective TODOs, not reported findings. No illustrative results, fabricated trajectories or predicted effect sizes are substituted for the missing measurements.

- **PF1: coverage and retention.** Plot all twelve trajectories across F1–F10, distinguishing feature and post-feature checkpoints. Show cumulative hidden coverage, open retained-contract losses and their at-risk denominators; mark unavailable evaluations rather than connecting through them. Required inputs are the frozen eligibility map, checkpoint pass vectors and loss/recovery records.
- **PF2: changeability and resources.** Show every allocated run's binary CR1 outcome with actual AI credits, including failed attempts. Separate pre-CR1 and CR1 consumption and identify missing observations and assisted runs. Required inputs are the frozen success/missingness rules, pre/post-CR1 pass vectors and traceable consumption records.
- **PF3: coverage–cost trade-off.** Plot final hidden coverage against actual pre-CR1 credits for every trajectory, distinguishing arms and deviations. Show run values before summaries; four runs per arm do not justify a smooth performance frontier or stable precision claims.

8 Discussion

8.1 What worked

The case is not evidence that agentic delivery is inherently ineffective. The research phase assembled a corpus and analysis in days; a human-only duration was not measured. The role agents' challenge killed a bad idea, five end-to-end agents, before any code was written. The agents refused to invent approvals, figures or company facts, and recorded the limits of their evidence. The MVP was built and reshaped in a day. These episodes included targeted human challenge and feedback; they are not a controlled comparison with human-only teams or proof that all short loops succeed.

8.2 Human judgement and reflective work

The participant supplied several kinds of reflective work: choosing what not to build, recognising the unusable replacement and deciding which earlier lessons mattered. Naur's theory-building account helps describe that retained understanding [68]. Beck's distinction between visible output and understanding supplies another framing [14]. These judgements were supplied by the author during the case. Whether agents could make comparable judgements remains open. The narrower experimental question is whether bounded post-feature correction preserves behaviour and later changeability.

Human sign-off also need not establish who formed a judgment. In Anthropic's randomised study, 50 agent-using professional analysts matched their agent's randomly assigned preference on 86% of four analytical choices [41, Study design; Results]. The preferences were deliberately strengthened in one farming-analysis task with one model; the rate is not an estimate for coding or Zava North Homes [41, Our study's limitations]. This challenges an assumption that retaining approval alone preserves independent judgment, not the case's reported decisions. Recording alternatives and a human's reasons before accepting an agent's recommendation is a proposed safeguard, not an evaluated remedy here [41, What this means].

8.3 Can another agent restore futures?

Beck's alternation suggests preserving the options consumed by feature execution [13, 15]. Darwin, AutoSaddler and SkillOpt already provide precedents for agent-guided correction; reflection itself is not the proposed novelty. The pilot instead asks what remains after a fixed repository-consolidation step, with H6 as one future-change probe. It does not test choosing when to interrupt delivery, changing product goals or restoring a deployment. Those questions, and matched human/agent steering, belong to the separate research agenda in the ancillary supplement.

8.4 Implications for practice

The case motivates protecting a usable increment, bounding the delivery horizon and keeping lessons retrievable or executable. The ancillary supplement retains the proposed practice checklist and framework-builder suggestions. Proportionality never overrides mandatory safety, privacy, approval or publication gates, and compact context views must retain their source evidence. The pilot does not test that practice package, an optimal human-intervention cadence or the preservation of expertise. Those remain separate questions.

9 Conclusion

The unit of progress in cumulative development is the system left for the next change. Knight Capital shows how a new deployment can activate an old fault. WyCash makes consolidation part of continued growth. Agent research adds a direct cross-feature test: in SWE-Maintain,

earlier changes affect later tasks even when conversation history is discarded. Its clean-history results make preserving behaviour a more precise target than treating all agent-written code as structurally deficient.

The instructions governing development need maintenance too. Repository studies show repeated additions and revisions, alongside pruning; controlled context-file comparisons show that more guidance can increase work without improving task resolution. Code, tests and instructions are all inherited state, but they require different measures of correctness and burden.

The exploratory testbed supplies a traceable delivery failure within this wider problem: 183 merged pull requests did not restore the owner's usable replacement. The case's five candidate mechanisms frame possible links among accumulated obligations, lost lessons, completion proxies and insufficient product-use feedback. Migration and deployment dependencies remain central to explaining that particular failure.

The planned twelve-trajectory pilot asks what scheduled consolidation leaves for later work. It separates final behavioural coverage from retained-contract loss, probes an unforeseen change and reports the resources consumed. Secondary metrics provide descriptive evidence; testing the proposed mechanisms would require further work. The pilot has not started; launch checks and registration remain pending. What matters is whether the agents' combined work preserves the behaviour and options needed by subsequent features.

Acknowledgements

The author thanks the colleagues and partners whose questions shaped this report.

Use of generative AI

GitHub Copilot with HVE Core agents supported development of the system under study. The case method distinguishes reported interface labels from recorded selectors; the deployed assistant exercise used `gpt-4.1-mini-2025-04-14`, not the coding-agent model. Microsoft 365 Copilot supported summarisation and reference discovery.

GitHub Copilot in VS Code assisted log analysis, mechanism naming and organisation, manuscript drafting and revision, bibliographic checks, public repository inspection, figure preparation and local validation. Scientific Agent Skills supplied developmental writing, evaluation and hypothesis guidance [48]. This citation records assistance tooling, not evidence for the case findings or consolidation hypothesis. AI assistance supplied neither an independent empirical coder nor replicated application or study results. The package README retains the revision-by-revision account and source-access limits. GitHub Copilot CLI assisted the 29 October source integration and scoped method-clarity revision. For the subsequent editorial and visual revision, Gemini 4 Argon proposed infographic concepts. Those suggestions were checked against the manuscript and adapted into editable Excalidraw conceptual diagrams; they are not experimental outputs or independent validation.

The author supplied the observations, selected the case and made the analytical judgements, and reports checking the repository figures and Azure expenditure and confirming the Beck paraphrase after reading the essay and watching the talk. Independent author checking of cited research statistics and the log-growth figure remains pending. Metadata checks do not establish claim support; source comparison does not independently validate a source's experiments. The author takes responsibility for the content. Further disclosure must follow the actual submission venue's policy, which has not been assessed here.

Data availability

The repository analysed is public at https://github.com/jmservera/ama; this report uses commit `29c6b512f5e68733bb2cc1882343db39f5eb2cad`. The log-measurement and figure-generation scripts are provided with the source. The ancillary file anc/case-evidence-matrix.json indexes O1–O5, including reported locators and missing release/execution links; it is not newly collected corroboration. The companion anc/supplementary-context.md retains practitioner material, external observations, proposed practices and deferred extensions. The ancillary file anc/repository-observations-2026-10-25.json records the additional repositories' source revisions, tracker counts, sampling boundary and claim locators without reporter identities or attachments. Live issue states can change independently of the pinned source commits. The ancillary experiment notes in HVE Core's Minimum Viable Experiment format are earlier proposals, not the tagged study record. The separate study kit is held in a private repository; it is not yet a public replication archive. The registered snapshot, sanitised data and exact run provenance will need an authorised deposit before a reproducibility claim can be made.

References

[1] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux. Spec-driven development as prescribed and as practiced. arXiv:2608.25202, 2026.

[2] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas. Manifesto for agile software development, 2001. https://agilemanifesto.org/.

[3] J. Young. Effective harnesses for long-running agents. Anthropic, 26 November 2025. https://www.anthropic.com/research/effective-harnesses-for-long-running-agents.

[4] A. Bacchelli and C. Bird. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 712–721. IEEE, 2013. doi:10.1109/ICSE.2013.6606617.

[5] L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi:10.1016/0005-1098(83)90046-8.

[6] S. Baltes et al. Guidelines for empirical studies in software engineering involving large language models. arXiv:2508.15503, 2025. Accepted in *Empirical Software Engineering*.

[7] G. Bavota, A. De Lucia, M. Di Penta, R. Oliveto, and F. Palomba. An experimental investigation on the innate relationship between quality and refactoring. *Journal of Systems and Software*, 107:1–14, 2015. doi:10.1016/j.jss.2015.05.024.

[8] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[9] K. Beck and C. Andres. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 2nd edition, 2004.

[10] K. Beck. `test && commit || revert`. Medium, 28 September 2018. https://medium.com/@kentbeck_7670/test-commit-revert-870bbd756864.

[11] K. Beck. *Tidy First? A Personal Exercise in Empirical Software Design*. O'Reilly Media, 2023.

[12] K. Beck. Augmented coding: Beyond the vibes. *Software Design: Tidy First?* (newsletter), 25 June 2025. https://tidyfirst.substack.com/p/augmented-coding-beyond-the-vibes.

[13] K. Beck. Why does development slow? *Software Design: Tidy First?*, 19 November 2025. Author's summary at https://kentbeck.com/summaries/why-does-development-slow/.

[14] K. Beck. Mathematicians, here's a way to think about your existential crisis. *Software Design: Tidy First?* (newsletter), 29 September 2026. https://newsletter.kentbeck.com/p/mathematicians-heres-a-way-to-think.

[15] K. Beck. Software engineering in the age of AI. Talk at Prodacity 2026, Nashville, TN, 25–27 August 2026. Video: https://www.youtube.com/watch?v=F8fBgDCf2Y4; transcript: https://www.rise8.us/videos/kent-beck-software-engineering-in-the-age-of-ai.

[16] J. Becker, N. Rush, E. Barnes, and D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv:2507.09089, 2025.

[17] T. Besker, A. Martini, and J. Bosch. Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work. In *Proceedings of the International Conference on Technical Debt (TechDebt '18)*, pages 105–114. ACM, 2018. doi:10.1145/3194164.3194178.

[18] C. Bird, V.-P. Ranganath, T. Zimmermann, N. Nagappan, and A. Zeller. Extrinsic influence factors in software reliability: A study of 200,000 Windows machines. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. ACM, 2014. https://www.microsoft.com/en-us/research/publication/extrinsic-influence-factors-in-software-reliability-a-study-of-200000-windows-machines/.

[19] B. Böckeler. Understanding spec-driven-development: Kiro, spec-kit, and Tessl. martinfowler.com, 15 October 2025. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html.

[20] M. Borg, D. Hewett, N. Hagatulah, N. Couderc, E. Söderberg, D. Graham, U. Kini, and D. Farley. Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability. *Empirical Software Engineering*, 31(6):161, 2026. doi:10.1007/s10664-026-10889-1.

[21] F. P. Brooks, Jr. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. doi:10.1109/MC.1987.1663532.

[22] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025. arXiv:2503.13657.

[23] W. Chatlatanagulchai, H. Li, Y. Kashiwa, B. Reid, K. Thonglek, P. Leelaprute, A. Rungsawang, B. Manaskasemsak, B. Adams, A. E. Hassan, and H. Iida. Agent READMEs: An empirical study of context files for agentic coding. arXiv:2511.12884v2, 9 August 2026 (first posted 2025). https://arxiv.org/abs/2511.12884v2.

[24] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. arXiv:2603.03823, 2026.

[25] A. Cockburn. *Crystal Clear: A Human-Powered Methodology for Small Teams.* Addison-Wesley, 2004.

[26] J. Cohen. *Best Kept Secrets of Peer Code Review.* Smart Bear Inc., 2006.

[27] W. Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings of OOPSLA '92 (Experience Report)*, pages 29–30. ACM, 1992. doi:10.1145/157709.157715.

[28] D. Delimarsky. Spec-driven development with AI: Get started with a new open source toolkit. The GitHub Blog, 2 September 2025. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/.

[29] P. Devanbu, T. Zimmermann, and C. Bird. Belief & evidence in empirical software engineering. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 2016. doi:10.1145/2884781.2884812.

[30] J. Ding, S. Long, C. Pu, et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents. arXiv:2512.12730, 2025.

[31] DORA. State of AI-assisted software development 2025. Google Cloud, 2025. https://dora.dev/dora-report-2025.

[32] S. G. Eick, T. L. Graves, A. F. Karr, J. S. Marron, and A. Mockus. Does code decay? Assessing the evidence from change management data. *IEEE Transactions on Software Engineering*, 27(1):1–12, 2001. doi:10.1109/32.895984.

[33] M. Fowler. Design stamina hypothesis. martinfowler.com, 20 June 2007. https://martinfowler.com/bliki/DesignStaminaHypothesis.html.

[34] M. Fowler. Some thoughts on LLMs and software development. martinfowler.com, 28 August 2025. https://martinfowler.com/articles/202508-ai-thoughts.html.

[35] J. Gall. *General Systemantics: An Essay on How Systems Work, and Especially How They Fail*. General Systemantics Press, Ann Arbor, MI, 1975.

[36] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN 978-0-201-63361-0. Publisher's preface and Adapter excerpt: https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610; https://www.informit.com/articles/article.aspx?p=1398600.

[37] D. Gautam, S. Garg, J. Jang, N. Sundaresan, and R. Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2503.07832.

[38] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents? arXiv:2602.11988v3, 29 September 2026. https://arxiv.org/abs/2602.11988v3.

[39] E. M. Goldratt and J. Cox. *The Goal: A Process of Ongoing Improvement*. North River Press, 1984.

[40] C. A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*. Reserve Bank of Australia, Sydney, 1975.

[41] A. Herdağdelen, H. Joren, and A. Tamkin. How AI agents shape experts' judgment calls. Anthropic research article, 27 October 2026; accessed 29 October 2026. https://www.anthropic.com/research/agents-judgment-calls.

[42] I. Herraiz, D. Rodriguez, G. Robles, and J. M. Gonzalez-Barahona. The evolution of the laws of software evolution: A discussion based on a systematic literature review. *ACM Computing Surveys*, 46(2):28, 2013. doi:10.1145/2543581.2543595.

[43] K. Hong, A. Troynikov, and J. Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, July 2025. https://trychroma.com/research/context-rot.

[44] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg. ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 598–608. PMLR, 2022. https://proceedings.mlr.press/v164/hoque22a.html.

[45] A. Hunt and D. Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999.

[46] Microsoft. HVE Core: HyperVelocity Engineering for GitHub Copilot. GitHub repository and documentation, accessed 24 October 2026. https://github.com/microsoft/hve-core; https://microsoft.github.io/hve-core/.

[47] Z. Jiang, F. Huang, H. Xing, X. Wu, Y. Gao, R. Cao, M. Wang, S. Liu, and Y. Li. Demystifying agent skills: Why they work–until they don't. arXiv preprint arXiv:2608.14036v1, 2026; accessed 29 October 2026. https://arxiv.org/html/2608.14036v1.

[48] T. Kassis, V. Agarwal, Y. He, D. Patel, and A. M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents. arXiv:2609.00065, 2026. doi:10.48550/arXiv.2609.00065.

[49] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer. HG-DAgger: Interactive imitation learning with human experts. arXiv:1810.02890v2, 2019 (first posted 2018).

[50] M. Kim, T. Zimmermann, and N. Nagappan. An empirical study of refactoring challenges and benefits at Microsoft. *IEEE Transactions on Software Engineering*, 40(7), 2014. doi:10.1109/TSE.2014.2318734.

[51] A. Kumar, Y. Bajpai, S. Gulwani, G. Soares, and E. Murphy-Hill. Why AI agents still need you: Findings from developer-agent collaborations in the wild. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025. arXiv:2506.12347v3, 9 October 2025.

[52] P. Laban, H. Hayashi, Y. Zhou, and J. Neville. LLMs get lost in multi-turn conversation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.06120v1, 9 May 2025.

[53] H.-P. Lee, A. Sarkar, L. Tankelevitch, I. Drosos, S. Rintel, R. Banks, and N. Wilson. The impact of generative AI on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–22. ACM, 2025. doi:10.1145/3706598.3713778.

[54] M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980. doi:10.1109/PROC.1980.11805.

[55] M. M. Lehman, J. F. Ramil, P. D. Wernick, D. E. Perry, and W. M. Turski. Metrics and laws of software evolution: The nineties view. In *Proceedings of the 4th International Software Metrics Symposium (METRICS '97)*. IEEE, 1997. doi:10.1109/METRIC.1997.637156.

[56] H. Li, H. Zhang, and A. E. Hassan. AIDev: Studying AI coding agents on GitHub. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2026. arXiv:2602.09185. doi:10.1145/3793302.3797249.

[57] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi:10.1162/tacl_a_00638.

[58] J. L. Lulla, M. Galster, J. M. Zhang, S. Baltes, and C. Treude. The impact of configuring agentic AI coding tools on build-vs-buy decisions: A study protocol. ESEM 2026, Registered Reports track. arXiv:2606.03907, 2026.

[59] Y. Ma, B. Kereopa-Yorke, and B. Schultz. Building to the test: Coding agents deliver what you check, not what you requested. arXiv preprint arXiv:2606.28430v1, 26 June 2026. https://arxiv.org/abs/2606.28430v1.

[60] R. C. Martin. Why hardcore technical focus still matters big time. *Product Engineers* podcast, 29 September 2026. https://productengineers.com/episodes/season2/why-hardcore-technical-focus-still-matters-big-time-uncle-bob/.

[61] S. Mhatre, Y. Bajpai, S. Gulwani, E. Murphy-Hill, and G. Soares. SWE-Sharp-Bench: A reproducible benchmark for C# software engineering tasks. In *International Conference on AI-powered Software (AIware)*, 2025. Accessed arXiv:2511.02352v2, 5 November 2025.

[62] Microsoft Research. Systematic debugging for AI agents: Introducing the AgentRx framework. First-party research blog, 12 March 2026; accessed 27 October 2026. https://www.microsoft.com/en-us/research/blog/systematic-debugging-for-ai-agents-introducing-the-agentrx-framework/.

[63] Microsoft Research. AutoSaddler: Automatic harness optimization with durable updates from agent execution traces. Publication-page abstract, page published 24 August 2026; accessed 27 October 2026. https://www.microsoft.com/en-us/research/publication/autosaddler-automatic-harness-optimization-with-durable-updates-from-agent-execution-traces-2/.

[64] Microsoft Research. SkillOpt: Agent skills as trainable parameters. First-party research blog, 30 June 2026; accessed 27 October 2026. https://www.microsoft.com/en-us/research/blog/skillopt-agent-skills-as-trainable-parameters/.

[65] S. Mohsenimofidi, M. Galster, C. Treude, and S. Baltes. Context engineering for AI agents in open-source software. arXiv:2510.21413v4, 5 February 2026 (first posted 2025). https://arxiv.org/abs/2510.21413v4.

[66] A. Murphy. If you thought the speed of writing code was your problem – you have bigger problems. Blog post, 17 March 2026. https://andrewmurphy.io/blog/if-you-thought-the-speed-of-writing-code-was-your-problem-you-have-bigger-problems.

[67] E. Murphy-Hill, J. Butler, and A. Savelieva. Adoption and impact of command-line AI coding agents: A study of Microsoft's early 2026 rollout of Claude Code and GitHub Copilot CLI. arXiv:2607.01418, 2026.

[68] P. Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985. doi:10.1016/0165-6074(85)90032-8.

[69] M. Nigh. How to write a great agents.md: Lessons from over 2,500 repositories. The GitHub Blog, 19 November 2025, updated 25 November 2025. https://github.blog/ai-and-ml/github-copilot/how-to-write-a-great-agents-md-lessons-from-over-2500-repositories/.

[70] R. Lopopolo. Harness engineering: Leveraging Codex in an agent-first world. OpenAI, 11 February 2026. https://openai.com/index/harness-engineering/.

[71] G. Orlanski, D. Roy, A. Yun, C. Shin, A. Gu, A. Ge, D. Adila, N. Roberts, F. Sala, and A. Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755, 2026.

[72] K. Beck, interviewed by G. Orosz. TDD, AI agents and coding with Kent Beck. *The Pragmatic Engineer* podcast, 11 June 2025. https://www.youtube.com/watch?v=aSXaxOdVtAQ.

[73] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):42, 2025. doi:10.1145/3697010.

[74] R. Pan, J. Zhang, L. Wu, Y. Zhang, H. Zhang, and L. Zhang. Do modifications made by coding agents make subsequent software maintenance harder? arXiv preprint arXiv:2610.14768v1, 10 October 2026.

[75] D. L. Parnas. Software aging. In *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, pages 279–287. IEEE, 1994. doi:10.1109/ICSE.1994.296790.

[76] A. Pavlenko, B. Crivat, B. Haynes, C. Curino, F. Psallidas, J. Slawinski, J. Freischuetz, L. Pereira Sanchez, M. Weimer, M. Demarne, M. Jasny, M. Gandhi, M. Bovykin, M. Milletari, P. Ghosh, Q. Bai, R. Ramakrishnan, R. Pandita, S. Matusevich, S. Venkataraman, S. Krishnan, M. T. Mahmood, T. Bang, V. Emani, X. Zhao, and Y. Zhu. Towards an AI software factory for data systems. arXiv:2609.36323v1, 28 September 2026. https://arxiv.org/abs/2609.36323v1.

[77] S. R. O. Peralta, F. Hoshi, H. Washizaki, N. Ubayashi, I. Kondo, Y. Higo, H. Mukai, N. Yoshida, K. Kusama, H. Tanaka, and Y. Fan. Why are agentic pull requests merged or rejected? An empirical study. arXiv:2605.22534, 2026.

[78] PhotoCraft contributors. PhotoCraft: Source, parity records and public tracker observations. GitHub, source commit `5bb8536`; retrieved 25 October 2026. https://github.com/storytold/photocraft/tree/5bb853654196eca807212ff8955ba2eb293071ea. Dated counts and claim locators in the ancillary repository-observation record.

[79] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield. Effective context engineering for AI agents. Anthropic, 29 September 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

[80] W. W. Royce. Managing the development of large software systems. In *Proceedings of IEEE WESCON*, pages 1–9, August 1970.

[81] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi:10.1007/s10664-008-9102-8.

[82] U.S. Securities and Exchange Commission. SEC charges Knight Capital with violations of market access rule. Press release 2013-222, 16 October 2013. https://www.sec.gov/newsroom/press-releases/2013-222.

[83] J. M. Servera Bondroit. AMA Case Study 48: research and solution design for the Zava Forward retrofit programme. GitHub repository, commit 29c6b51, 24 October 2026. https://github.com/jmservera/ama.

[84] P. Ralph et al. Empirical standards for software engineering research. arXiv:2010.03525v2, 2021 (first posted 2020). ACM SIGSOFT Empirical Standards: https://www2.sigsoft.org/EmpiricalStandards/.

[85] A. Simkute, L. Tankelevitch, V. Kewenig, A. E. Scott, A. Sellen, and S. Rintel. Ironies of generative AI: Understanding and mitigating productivity loss in human-AI interaction. *International Journal of Human–Computer Interaction*, 41(5):2898–2919, 2025. doi:10.1080/10447318.2024.2405782.

[86] A. Sinha, A. Arun, S. Goel, S. Staab, and J. Geiping. The illusion of diminishing returns: Measuring long horizon execution in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.09677v3, 13 March 2026.

[87] M. Strathern. 'Improving ratings': Audit in the British university system. *European Review*, 5(3):305–321, 1997.

[88] L. Tankelevitch, V. Kewenig, A. Simkute, A. E. Scott, A. Sarkar, A. Sellen, and S. Rintel. The metacognitive demands and opportunities of generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pp. 1–24. ACM, 2024. doi:10.1145/3613904.3642902.

[89] A. Tornhill and M. Borg. Code red: The business impact of code quality. A quantitative study of 39 proprietary production codebases. In *Proceedings of the International Conference on Technical Debt (TechDebt '22)*, pages 11–20. ACM, 2022. doi:10.1145/3524843.3528091.

[90] M. Waseem, A. Ahmad, K.-K. Kemell, J. Rasku, S. Lahti, K. Mäkelä, and P. Abrahamsson. Vibe coding in practice: Flow, technical debt, and guidelines for sustainable use. arXiv:2512.11922, 2025.

[91] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi:10.1007/978-3-642-29044-2.

[92] W. Yan. Don't build multi-agents. Cognition blog, 12 June 2025. https://cognition.ai/blog/dont-build-multi-agents.

[93] C. Yang, H. He, T.-C. Cheng, J. Butler, E. Murphy-Hill, C. Badea, and C. Bird. (When) should a software factory be lights-off? Benchmarking code review agents on bug detection. arXiv preprint arXiv:2610.13668v1, 9 October 2026. https://arxiv.org/abs/2610.13668v1.

[94] H. Yu, L. Liu, X. Jiang, Y. Jia, S. Wang, P. Qian, and Y. Chen. Habituation at the gate: Rising approval and declining scrutiny in human review of AI agent code. KDD 2026 Workshop on Agentic Software Engineering, 2026. arXiv:2606.22721.

[95] M. Zhang, O. Press, W. Merrill, A. Liu, and N. A. Smith. How language model hallucinations can snowball. arXiv preprint arXiv:2305.13534v1, 22 May 2023. doi:10.48550/arXiv.2305.13534.

[96] Z. Zhong, A. Raghunathan, and N. Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.20270.

A Case rival-explanation register

The main interpretation is in Section 5.7. The register preserves recorded rivals separately from untested alternatives. A limit is not grounds for ruling an explanation out. Score/deployment correspondence remains unverified (Section 3.6).

Table 11: Rival explanations with direct case records.

Explanation	Case support	Counter-evidence or limits	Treatment
Migration decision	ADR 0006 (issue #196) froze feature work in the MVP and prioritised a new layout.	Health and restart checks passed, not usability.	Migration and coordination changed together; autonomy alone is not identified.
Capstone rubric	Research and documentation are rewarded; full completeness marks coexist with blocked sign-in.	A working demonstrator is required; release/scope linkage is pending.	Possible incentive, not a measured effect.
Owner overrides and priorities	Committed RPI records, full gates and delegated merging; PR #445 required an exception.	The MVP used agents successfully; HVE Core defaults retain human stops.	Owner choices are part of the case.
Model and tool changes	Versions changed during the study.	No matched rerun under fixed versions.	Actual session configuration remains partly unknown.
CI and cloud budgets	Issue #413 moved validation locally; costs prompted a higher ceiling and reduced resources.	Merges continued without established deployed usability.	More budget is not proven to resolve delivery.
Identity and sign-in setup	Missing registration/staff bindings; issue #349 distinguishes redirect from sign-in.	Those steps worked on 12 October.	Concrete configuration obstacle, not proof of code decay or lost knowledge.

Table 12: Additional plausible rivals and unperformed discriminating checks.

Explanation	Reason considered	Counter-evidence or limits	Evidence still needed
External services and deployment drift	Model/identity configuration differs from the MVP.	No provider outage is established; health checks were narrower.	Versioned deployment/configuration and dependency-health records for the journey.
Scope expansion	RAI and consumption criteria added to issues; further resident-preview obligations.	Expansion predates autonomy; some controls are justified.	Original/added scope, sequencing and time-to-outcome comparison.
Inadequate acceptance tests	Checks passed without completed real-user sign-in; scorer read artefacts.	Bounded, honest tests need not be poor tests.	Capability-to-task-oracle mapping and execution evidence.
Rate and context limits	Parallel sessions and a large read path.	No runtime rate-limit, truncation or attention dataset analysed.	Actual inputs and blocked/retried request events.
Specification ambiguity	Selective review of long, cross-referenced requirements.	Important domain/consent decisions survived; ambiguity not independently coded.	Independent interpretation and conflicting acceptance examples.
Ordinary prototype debt	Rapid MVP followed by restructuring and rebuilding.	MVP worked intermittently; no comparable human-led migration.	Failure analysis distinguishing inherited debt from introduced faults.

B Candidate-mechanism evidence audits

These are operational boundaries for this paper, not validated necessary and sufficient conditions for all agentic development. The indicators refer to Table 1 and the case chronology. Experimen-

tal implications state what evidence would discriminate an explanation; the tagged experiment does not manipulate the five candidates separately.

B.1 Specification inflation

Element	Audit
Definition	Requirements and acceptance obligations expand faster than independently validated behaviour, without evidence that the added scope is needed for the next usable increment. Length alone is insufficient.
Case indicators	O3: BRD/PRD growth, 192 issues and cross-cutting additions; the smallest resident preview had thirteen criteria.
Counter-evidence	Growth began before autonomy and sometimes followed the owner's feedback; regulatory and consent controls can justify detail.
Rival explanations	Genuine scope expansion, specification ambiguity, capstone incentives and an ambitious migration rather than agent-driven inflation.
Experimental implication	Compare smaller requirements horizons at matched scope and resources. The current fixed-suite experiment cannot identify this mechanism, even if consolidation helps.

B.2 Proxy capture

Element	Audit
Definition	A completion proxy becomes a decision criterion and remains positive when an independent test of the intended outcome is negative. Proxy use alone does not establish capture.
Case indicators	O2–O3: a 5/5 score and blocked sign-in coexist in the chronology; release/scope linkage is pending. Mandatory artefact presence and the evidence record for PR #445 are separate indicators.
Counter-evidence	Agents explicitly reported unmet acceptance; neither fabricated approval nor deliberate deception is established.
Rival explanations	An inadequate scorer or acceptance oracle, different evaluation targets, and a deployment failure invisible to repository review.
Experimental implication	Disagreement between visible completion and hidden behavioural tests would expose proxy limits. It would not by itself establish intentional test gaming or explain why a gate was chosen.

B.3 Learning evaporation

Element	Audit
Definition	A recorded lesson relevant to a later decision is absent from, unusable on, or not applied through that decision's effective context path, and the associated problem recurs. A missing citation alone is insufficient.
Case indicators	O4: simplicity references cease; working sign-in setup is not preserved; state and intent documents reach about 139,000 words.
Counter-evidence	Important domain decisions survived; the lesson could have been known but deferred. Actual session attention was not measured.
Rival explanations	Deliberate migration sequencing, deployment-specific configuration, context limits, or conflicting requirements rather than lost knowledge.
Experimental implication	Trace actual inputs and repeated faults. B–C cannot isolate memory quality because DESIGN and REVIEW notes and acting permissions differ. A matched-context study would be needed.

B.4 Process-gate inflation

Element	Audit
Definition	Additional procedural obligations increase per-change burden without demonstrated proportional improvement in the targeted outcome. More gates alone are not evidence that their net benefit is negative.
Case indicators	O5: full RPI and validation on each correction/rebase; hosted-minute rationing; explicit exception for PR #445.
Counter-evidence	Checks caught bounded defects and enforced evidence integrity. Gate duration and human review effort were not measured per PR.
Rival explanations	Legitimate safety and privacy requirements, budget constraints, new deployment complexity and owner-selected risk tolerance.
Experimental implication	Measure gate time and accepted outcomes under matched risk. The pilot's resource description concerns the package, not the effect of removing this repository's required gates.

B.5 Feedback starvation

Element	Audit
Definition	Independent outcome-level feedback is insufficient to correct delivery decisions despite abundant artefact-level validation. The defining gap is between the evaluated proxy and intended outcome, not test count alone.
Case indicators	O1–O2 and O5: working MVP frozen; replacement unusable; passing checks and continued approvals did not restore a completed user task.
Counter-evidence	Human oversight continued; agents surfaced blockers; engineering checks provided real, though narrower, feedback.
Rival explanations	Identity and service configuration, migration priority, incomplete acceptance coverage, prototype debt or insufficient resources.
Experimental implication	Hidden retention tests and CR1 supply behavioural feedback to the evaluator. The experiment does not test live deployment recovery, product selection or a human-use checkpoint cadence.

C Measures and lexicon

Log entries. The knowledge-base log (`docs/reference/log.md`) was split into entries at each second-level heading that begins with a date. An entry belongs to the date in its heading. Word counts use whitespace tokenisation of the entry, heading included.

Hedging phrases. Each entry was searched, case-insensitively and on word boundaries, for these patterns: *unmet*; *not accepted*; *not inferred*; *is inferred*; *remain(s) open*; *remain(s) unproved*; *remain(s) unverified*; *not claimed*; *not established*; and *no* followed, within forty letters, spaces, commas, slashes or hyphens, by *is* or *was* and then *claimed* or *inferred*. Density is matches per 1,000 words.

Reference tracing. An entry cites the simplicity requirements if it contains an identifier from SA-001 to SA-010 or the word *simplicity*. It cites the development profile if it contains *deploymentProfile*, *development profile* or *allowPublicNetworkAccess*.

Repository counts. Issue and pull-request counts come from the GitHub search API on 24 October 2026 (for example, `is:pr is:merged merged:>=2026-10-15`). Document sizes are whitespace word counts of the files at commit `29c6b51`.

Proposed measures. *Time to first human use*: elapsed time from the start of a delivery cycle to the first time a person uses the running increment for its purpose. *Age of the last usable increment*: time since a person last verified that the deployed system could be used for its main journey. *Behaviour-changing merge ratio*: the share of merged pull requests that change what a user can observe in the running increment. *Cost per accepted outcome*: total model, tool and cloud cost divided by the number of outcomes a human accepted in the same period.

D Consolidation step prompt

Proposed prompt for the consolidation step of arm B (Section 6). Braces mark parameters set by the experiment runner.

```
Do not add or change features. Your job is to leave the code better than
you found it without changing its behaviour.

1. Run ‘make check‘, then read reports/quality.md (functions above
   complexity 10, duplicated blocks, lint warnings, file sizes).
2. Review the last feature’s diff (‘git diff feature-start feature-end‘,
   tags the runner sets around each feature step) and the hotspots
   against this checklist: duplicated logic; long or complex functions;
   unclear names; dead code; responsibilities in the wrong place, or
   rules from one feature leaking into another; tests that check
   implementation details instead of behaviour.
3. Make small, behaviour-preserving changes. Apply every change through
   ‘./tcr "<message>"‘: it runs all tests and commits if they pass, or
   reverts if they fail.
4. Never modify or delete files in tests/examples/. You may add tests that
   pin current behaviour before you refactor.
5. Rewrite DESIGN.md in at most 300 words: the current structure, where
   each business rule lives, and known debts. Replace; do not append.
6. Stop when the quality budget is met (no function above complexity 10,
   duplicated lines below 3%, zero warnings in the configured lint rules)
   or when your time budget ({consolidation_budget}) is used.
7. Finish with a summary of at most five lines: what you changed and what
   debt remains.
```